

ORGANIZATIONAL BRAIN

MASTER PROJECT CONSTITUTION

The Authoritative Reference for Project Development

Document Version: 2.0

Document Type: Project Constitution - FULL ROADMAP FROM A to Z

Date: September 2026

Project: Organizational Brain - Enterprise Intelligence Platform

Team: Faculty of Computers and Data Science - Graduation Project 2026-2027

AUTHORITY STATEMENT

This document is the highest-level reference for the Organizational Brain project. All development decisions, architectural choices, and implementation work must align with this Constitution. If implementation contradicts this Constitution, the Constitution must be reviewed and formally amended—never silently violated.

TABLE OF CONTENTS

1. [Executive Summary](#)
2. [Project Vision](#)
3. [Product Definition](#)
4. [Architecture](#)
5. [Three-Layer Intelligence Model](#)
6. [General Brain](#)
7. [Domain Modules](#)
8. [Organization Intelligence](#)
9. [Architectural Invariants](#)
10. [Knowledge Lifecycle](#)
11. [Legal Domain](#)
12. [Case Model](#)
13. [WhatsApp Model](#)
14. [CRM Model](#)
15. [AI Employee Model](#)
16. [Autonomy Model](#)
17. [Governance](#)
18. [Permissions](#)
19. [Data Architecture](#)
20. [AI Architecture](#)
21. [Evaluation Strategy](#)
22. [Software Analysis Requirements](#)
23. [UI/UX Principles](#)
24. [Security](#)
25. [Testing](#)
26. [Documentation Governance](#)
27. [Change Management](#)
28. [Ring Architecture](#)
29. [Cross-Functional Collaboration](#)
30. [Ring Entry/Exit Criteria](#)

31. [V1 Definition](#)
 32. [Future Domain Strategy](#)
 33. [Team Operating Model](#)
 34. [Source of Truth Hierarchy](#)
 35. [Master Roadmap](#)
 36. [Definition of Done](#)
 37. [Glossary](#)
-

EXECUTIVE SUMMARY

The Core Question

Can one reusable General Brain become intelligent inside different organizations by combining:

- GENERAL INTELLIGENCE
- DOMAIN KNOWLEDGE
- ORGANIZATIONAL EXPERIENCE?

The Answer

Yes. By architecting the system as three distinct layers—General Brain, Domain Intelligence, and Organization-Specific Intelligence—we create a reusable platform that can be deployed across industries without rebuilding the core intelligence infrastructure.

The Approach

We are building an **Enterprise Intelligence Platform**, not a chatbot, not a RAG application, not a CRM. The Organizational Brain is an intelligence layer that observes, learns, validates, and assists—governed by human authority at every step.

The First Domain

Legal (Egyptian Civil Law) is our first domain implementation. Legal is not the final product—it is the proof that the General Brain architecture works in a real, high-stakes environment.

The Roadmap

This Constitution provides a complete, ring-based implementation roadmap from foundation through Legal V1 to future domain expansion. Each Ring represents a complete, working vertical capability spanning Frontend, Backend, AI, and Data Science.

PROJECT VISION

What We Are Building

ONE GENERAL ORGANIZATIONAL BRAIN

A reusable intelligence infrastructure that can be specialized by domain and customized by organization.

What We Are NOT Building

- X A chatbot
- X A generic AI assistant

- X A RAG application
- X A CRM
- X A collection of unrelated AI agents
- X A Legal AI chatbot
- X A separate Brain for every industry

The Long-Term Vision

ONE BRAIN

-> MULTIPLE DOMAINS

-> MULTIPLE ORGANIZATIONS

-> ORGANIZATION-SPECIFIC INTELLIGENCE

The Core Value Proposition

Every organization has knowledge. We give it a brain.

Knowledge stops being trapped inside individual employees. It becomes organizational memory—governed, validated, and continuously improving.

PRODUCT DEFINITION

Core Product Architecture

GENERAL BRAIN

↓

DOMAIN / INDUSTRY INTELLIGENCE

↓

ORGANIZATION INTELLIGENCE

↓

ORGANIZATIONAL BRAIN

The Fundamental Equation

General Brain

+

Domain Knowledge

+

Organization-Specific Knowledge

+

Organization Systems

+

Governed Learning

=

Organization-Specific Intelligence

Architectural Invariant

The General Brain is reusable.

A Domain Module does NOT create a new Brain.

An Organization does NOT create a new Brain.

This principle must never be violated without explicit architectural approval.

ARCHITECTURE

Architectural Principles

1. ONE GENERAL BRAIN

The General Brain is built once and reused across industries. Domain-specific logic is NOT built into the General Brain.

2. THREE-LAYER SEPARATION

- **Layer 1: General Brain** - Reusable intelligence infrastructure
- **Layer 2: Domain Intelligence** - Domain-specific knowledge and capabilities
- **Layer 3: Organization Intelligence** - Organization-specific context and learning

These layers must remain architecturally distinct.

3. OBSERVATION != KNOWLEDGE

Not every message, document, or observation becomes organizational knowledge. The learning loop explicitly distinguishes: - Raw observation - Processed understanding - Knowledge candidate - Validated knowledge

4. HUMAN GOVERNANCE

The organization controls: - Permissions - Validation authority - Autonomy levels - Access control

The Brain recommends and assists. Humans validate and govern.

5. CASE ISOLATION

Cases are first-class objects. Knowledge from Case A must NEVER leak into Case B, even for the same client.

6. GRADUAL AUTONOMY

Autonomy is earned, not assumed: - Level 1: Observe - Level 2: Assist - Level 3: Execute with approval - Level 4: Authorized routine execution

7. CRM != BRAIN

The Brain is the intelligence layer. CRM is an operational system. The Brain observes and learns from CRM but does not replace it.

8. RING-BASED DEVELOPMENT

Each Ring is a complete, working vertical capability. No “frontend phase” followed by “backend phase.” Each Ring spans the entire stack where relevant.

9. REUSABILITY FIRST

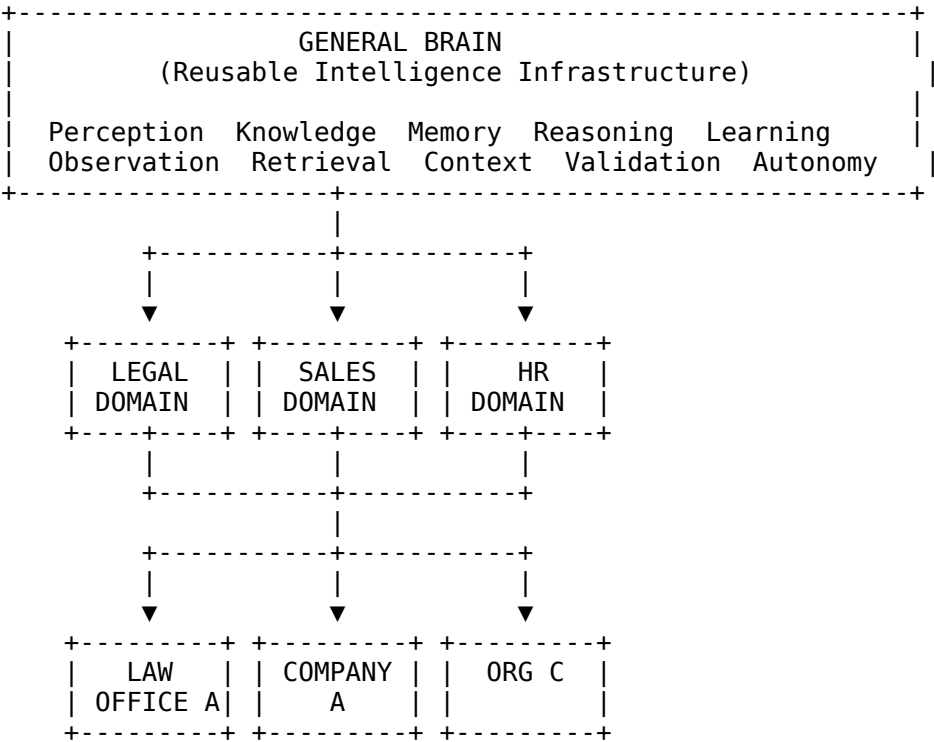
Every implementation decision is evaluated: “Can this be reused by another domain?” If YES, build as General Brain infrastructure.

10. PROVENANCE & AUDIT

Every knowledge object must have: - Source - Validation history - Provenance chain - Access permissions

THREE-LAYER INTELLIGENCE MODEL

Visual Model



Layer 1: General Brain [GB]

Purpose: Reusable intelligence infrastructure

Capabilities: - Perception Engine - Observation Engine - Knowledge Engine - Memory / Storage Engine - Retrieval Engine - Reasoning Engine - Context Engine - Learning Engine - Validation / Governance Engine - Conflict Detection Engine - Provenance Engine - Permission / Authorization Engine - Audit Engine - Insight Engine - Workflow / Action Engine - Autonomy Controller

Classification: [GB] GENERAL BRAIN

Layer 2: Domain Intelligence [DOMAIN]

Purpose: Domain-specific knowledge and capabilities

Examples: - Legal (Egyptian Civil Law) - Sales - HR - Finance - Customer Support - Real Estate

Provides: - Domain Knowledge - Domain Ontology / Taxonomy - Domain terminology - Domain workflows - Domain-specific reasoning patterns - Domain-specific tools - Domain-specific engines (where genuinely required) - Domain-specific evaluation - Domain-specific constraints - Domain-specific document types

Classification: [DOMAIN] DOMAIN-SPECIFIC

Layer 3: Organization Intelligence [ORG]

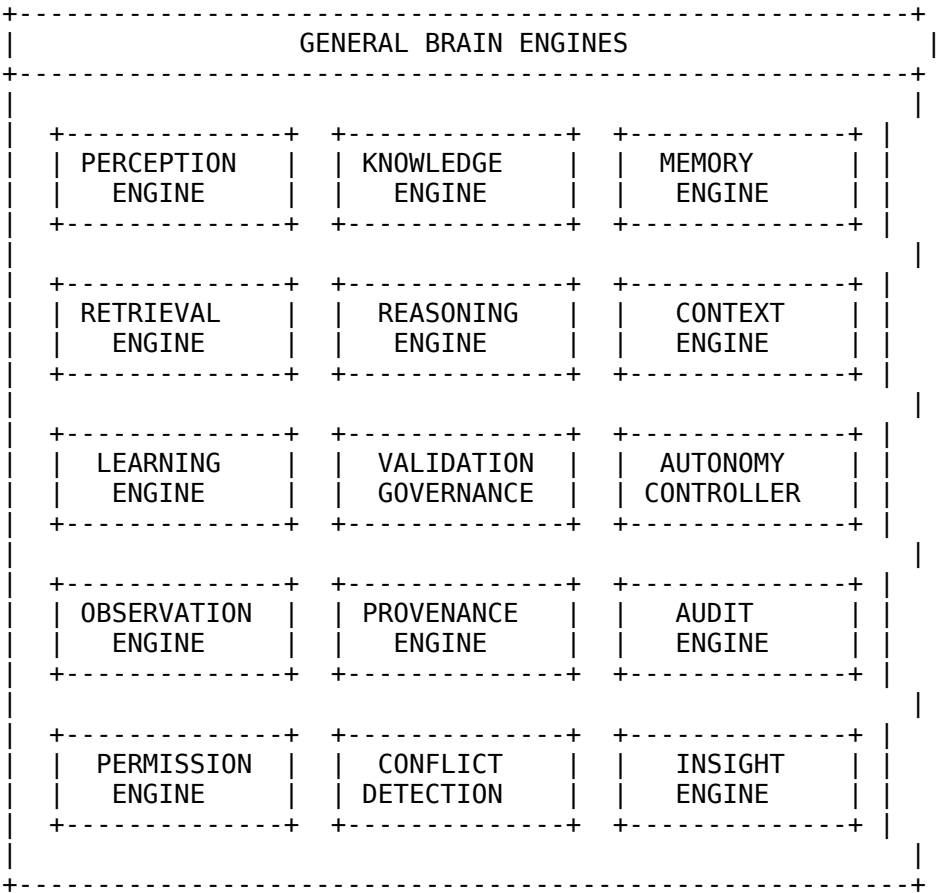
Purpose: Organization-specific context and learning

Provides: - Organization policies - Workflows - Roles - Permissions - Employees - Internal procedures - Organizational knowledge - Historical work - Documents - Cases - Conversations - Decisions - Outcomes - Business rules - Templates - Existing systems - CRM data - Operational patterns - Human corrections - Validated organizational knowledge

Classification: [ORG] ORGANIZATION-SPECIFIC

GENERAL BRAIN

Engine Map



Engine Descriptions

Perception Engine [GB]

Purpose: Raw input ingestion and initial processing **Inputs:** Text, audio, images, documents, system events **Outputs:** Structured observations **Reusable:** Yes, across all domains

Observation Engine [GB]

Purpose: Continuous monitoring of organizational activity **Inputs:** System events, user actions, external triggers **Outputs:** Observation stream **Reusable:** Yes, across all domains

Knowledge Engine [GB]

Purpose: Transform observations into knowledge candidates **Inputs:** Observations, context, existing knowledge **Outputs:** Knowledge candidates **Reusable:** Yes, across all domains

Memory / Storage Engine [GB]

Purpose: Multi-store knowledge persistence **Stores:** Relational, Vector, Graph, Object Storage **Reusable:** Yes, across all domains

Retrieval Engine [GB]

Purpose: Retrieve relevant knowledge for queries **Inputs:** Query, context, permissions **Outputs:** Ranked knowledge objects **Reusable:** Yes, across all domains

Reasoning Engine [GB]

Purpose: Multi-step reasoning with citations **Inputs:** Retrieved knowledge, query, context **Outputs:** Reasoned responses with confidence **Reusable:** Yes, across all domains

Context Engine [GB]

Purpose: Maintain and resolve contextual boundaries **Inputs:** Organization, domain, case, user **Outputs:** Contextual scope **Reusable:** Yes, across all domains

Learning Engine [GB]

Purpose: Identify learning opportunities and score candidates **Inputs:** Knowledge candidates, validation history **Outputs:** Learning signals **Reusable:** Yes, across all domains

Validation / Governance Engine [GB]

Purpose: Enforce human validation and governance rules **Inputs:** Knowledge candidates, permissions, validation authority **Outputs:** Validated knowledge or rejection **Reusable:** Yes, across all domains

Autonomy Controller [GB]

Purpose: Manage autonomy progression and execution permissions **Inputs:** Autonomy level, permissions, task type **Outputs:** Execution approval or rejection **Reusable:** Yes, across all domains

Provenance Engine [GB]

Purpose: Track source and validation history of all knowledge **Inputs:** Knowledge objects, validation events **Outputs:** Provenance chain **Reusable:** Yes, across all domains

Permission / Authorization Engine [GB]

Purpose: Enforce access control and authorization **Inputs:** User, role, resource, action **Outputs:** Allow or deny **Reusable:** Yes, across all domains

Audit Engine [GB]

Purpose: Log all significant system events **Inputs:** All system events **Outputs:** Audit trail **Reusable:** Yes, across all domains

Conflict Detection Engine [GB]

Purpose: Detect conflicts between knowledge objects **Inputs:** Knowledge objects, rules **Outputs:** Conflict reports **Reusable:** Yes, across all domains

Insight Engine [GB]

Purpose: Generate organizational insights from activity **Inputs:** Activity logs, knowledge, patterns **Outputs:** Insight reports **Reusable:** Yes, across all domains

Workflow / Action Engine [GB]

Purpose: Execute or recommend workflow actions **Inputs:** Workflow definitions, context, autonomy level **Outputs:** Action execution or recommendation **Reusable:** Yes, across all domains

DOMAIN MODULES

Legal Domain (First Implementation)

Domain Knowledge [DOMAIN]

Scope: Egyptian Civil Law **Components:** - Legal corpus (laws, regulations, precedents) - Legal concepts and terminology - Legal taxonomy - Legal procedures - Legal document structures - Historical legal cases (public) - Legal reasoning patterns

Domain-Specific Engines [DOMAIN]

Where genuinely required: - Legal Retrieval (domain-specific ranking) - Legal Classification (document type classification) - Legal Document Understanding (clause extraction) - Legal Similarity (case similarity) - Legal Source Reasoning (citation reasoning)

NOT separate Legal Brain. These are domain adapters for General Brain engines.

Domain Workflows [DOMAIN]

- Case intake
- Document review
- Legal research
- Client consultation
- Court filing
- Contract review

Future Domains

After Legal V1 proves the architecture, the system should be capable of adding:

- Sales
- Customer Support
- HR
- Finance
- Real Estate
- Other industries/domains

Principle: The same General Brain capabilities remain reusable. New domains reuse existing infrastructure.

ORGANIZATION INTELLIGENCE

Organization Context [ORG]

Three-Level Knowledge Distinction

1. General Domain Knowledge - Example: Egyptian Civil Law - Scope: Public, shared across all organizations - Classification: [DOMAIN]

2. Organization Knowledge - Example: How Law Office X handles civil cases - Scope: Specific to one organization - Classification: [ORG]

3. Case Knowledge - Example: Everything about Client Y's Case Z - Scope: Specific to one case - Classification: [ORG] - **Critical:** Case knowledge MUST NEVER leak across cases

Organization Configuration [ORG]

Configurable Elements

- Internal policies
- Workflows
- Roles and permissions
- Employees and teams
- Templates
- Business rules
- Integration endpoints
- Validation authority
- Autonomy levels per role

Principle

CONFIGURE THE BRAIN, NOT REBUILD THE BRAIN.

ARCHITECTURAL INVARIANTS

These are rules that developers must NOT violate without explicit architectural approval.

Core Invariants

- 1. General Brain capabilities must remain reusable.**
 - No domain-specific logic in General Brain components without architectural review
- 2. Domain logic must not unnecessarily leak into General Brain components.**
 - Domain adapters are the only acceptable mechanism
- 3. Organization-specific logic must not become hardcoded domain logic.**
 - Organization configuration must remain configurable
- 4. Legal must remain a Domain Module, not a separate Brain.**
 - Legal is the first domain, not a separate product
- 5. AI Employees operate on top of the Brain.**
 - AI Employees are role-specific workers, not separate intelligence systems
- 6. CRM is an operational system, not organizational memory itself.**
 - The Brain observes and learns from CRM but does not replace it
- 7. Observation is not automatically Knowledge.**
 - The learning loop must distinguish observation from knowledge
- 8. Candidate Knowledge is not Trusted Knowledge.**
 - Human validation is required before promotion
- 9. Human validation is required according to governance policy.**

- The Brain cannot auto-validate its own knowledge
- 10. **AI autonomy must be permission-controlled.**
 - Autonomy levels are organization-controlled
- 11. **Every important knowledge item must maintain provenance.**
 - Source, validation history, and chain must be tracked
- 12. **Conflicts must be detectable and traceable.**
 - Conflict detection must be implemented and auditable
- 13. **Case-specific knowledge must remain separated from general organizational knowledge.**
 - Case isolation is mandatory
- 14. **Multiple cases belonging to the same client must never be mixed.**
 - Case isolation applies even for the same client
- 15. **Permissions must be enforced server-side.**
 - Frontend permissions are presentation controls, not security controls
- 16. **Frontend permissions are presentation controls, not security controls.**
 - Security must be enforced at the server level
- 17. **AI outputs must be auditable where appropriate.**
 - Important AI decisions must be logged
- 18. **No Ring may silently redefine shared architecture.**
 - Architectural changes require documentation and review
- 19. **No team should duplicate an existing General Brain capability.**
 - Reuse is mandatory before building new
- 20. **New Domain Modules should reuse existing Brain infrastructure whenever possible.**
 - Domain expansion should not require Brain rebuild

Violation Process

If an invariant must be violated:

1. Document the violation in an Architecture Decision Record (ADR)
2. Explain why the violation is necessary
3. Propose alternatives considered
4. Get architectural approval
5. Update this Constitution if the violation becomes permanent

KNOWLEDGE LIFECYCLE

Formal Knowledge Lifecycle

Raw Observation
v
Structured Observation
v
Extracted Fact
v
Case / Organization Update
v
Knowledge Candidate
v
Validation
v
Trusted Knowledge
v
Persistent Memory
v
Retrieval
v
Reasoning
v
Decision Support / Action

Critical Distinctions

Not everything the Brain observes becomes Knowledge.

The system must maintain this distinction architecturally.

Knowledge Object Metadata

Every knowledge object must include:

- **Source** - Where the knowledge came from
- **Timestamp** - When it was created
- **Actor** - Who or what created it
- **Confidence** - System confidence score
- **Validation Status** - candidate, validated, rejected
- **Scope** - General, Domain, Organization, Case
- **Provenance** - Chain of sources and validations
- **Version** - Version tracking for updates
- **Conflict State** - Whether conflicts exist

Knowledge States

1. **Raw Observation** - Unprocessed input
 2. **Structured Observation** - Parsed and categorized
 3. **Extracted Fact** - Meaningful information extracted
 4. **Knowledge Candidate** - Proposed knowledge awaiting review
 5. **Trusted Knowledge** - Human-validated organizational knowledge
 6. **Archived Knowledge** - Deprecated but retained for audit
-

LEGAL DOMAIN

Legal as First Domain

LEGAL IS NOT A SEPARATE BRAIN.

Legal is the first domain specialization of the General Brain.

The architecture must remain reusable for future domains.

Legal Domain Knowledge [DOMAIN]

Scope: Egyptian Civil Law

Components: - Legal corpus (laws, regulations, precedents) - Legal concepts and terminology - Legal taxonomy - Legal procedures - Legal document structures - Historical legal cases (public) - Legal reasoning patterns

Legal-Specific Engines [DOMAIN]

Where genuinely required: - Legal Retrieval (domain-specific ranking) - Legal Classification (document type classification) - Legal Document Understanding (clause extraction) - Legal Similarity (case similarity) - Legal Source Reasoning (citation reasoning)

These are domain adapters for General Brain engines, not separate “Legal Brain” engines.

Legal Workflows [DOMAIN]

- Case intake
 - Document review
 - Legal research
 - Client consultation
 - Court filing
 - Contract review
-

CASE MODEL

Case Hierarchy

Organization
v
Client
v
Case A
v
Conversations
Documents
Events
Facts
Decisions
Knowledge

and:

Client
v
Case B
v
Different Conversations
Different Documents
Different Facts
Different Knowledge

Case Isolation Rules

- 1. Cases are first-class objects.**
 - Every case has a unique identifier
 - Cases belong to clients
 - A client may have multiple cases
- 2. Cases must remain isolated.**
 - Knowledge from Case A must NEVER leak into Case B
 - This applies even for the same client
 - Case isolation is a critical security requirement
- 3. Conversations belong to cases.**
 - WhatsApp conversations must be associated with the correct case
 - Case context must be maintained throughout conversation processing
- 4. Three-Level Knowledge Distinction in Cases:**
 - General Legal Knowledge (shared across all organizations)
 - Organization Legal Knowledge (specific to one law office)
 - Case-Specific Knowledge (specific to one case)

Case State Model

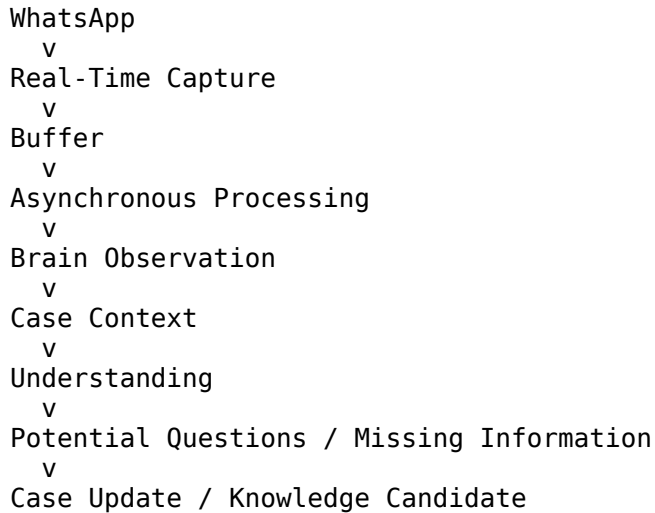
New -> Active -> In Progress -> Resolved -> Archived

WHATSAPP MODEL

WhatsApp as First-Class Integration

WhatsApp is the primary client communication channel in the initial Legal deployment.

Communication Architecture



Key Principles

1. **Real-time capture with async processing**
 - Messages are captured in real-time
 - Processing happens asynchronously to avoid blocking
2. **Not every message becomes knowledge**
 - Conversation != Knowledge
 - Human validation remains part of governance
3. **Case context is maintained**
 - Messages are associated with the correct case
 - Client identification is performed
4. **Conversation structure understanding**
 - The Brain understands conversation flow
 - Facts are extracted from conversations

Telephone Conversations

If a lawyer speaks to a client by phone:

1. The employee/lawyer updates the system manually
2. The Brain processes the update
3. It may identify useful information
4. It may propose knowledge
5. It may identify conflicts
6. But it must not blindly treat every update as trusted knowledge

CRM MODEL

CRM Principle

CRM = operational system. Brain = intelligence and memory layer.

Two Deployment Models

OPTION A: Existing CRM

- Integrate with organization's existing CRM
- Observe CRM events
- Learn from CRM data
- Provide intelligence back to CRM

OPTION B: No Suitable CRM

- Provide standardized domain-specific CRM capability
- Legal CRM for law offices
- Organization configures to their needs
- CRM becomes observation source and intelligence target

Scalability Principle

Do not create a completely new CRM from scratch for every customer.

Instead:

STANDARDIZED DOMAIN CRM
+
ORGANIZATION CONFIGURATION

This preserves scalability.

AI EMPLOYEE MODEL

AI Employees Are Not Separate Intelligence Systems

AI Employees are role-specific workers built on top of:

- General Brain
- Domain
- Organization Context
- Permissions

Example AI Employees

- Legal AI Employee
- Sales AI Employee
- HR AI Employee

Reuse Principle

AI Employees should reuse:

- Memory
- Knowledge
- Reasoning
- Retrieval
- Permissions
- Governance
- Organization Context

Do not duplicate Brain infrastructure unnecessarily for each AI Employee.

AUTONOMY MODEL

Autonomy as Progressive

Define the Brain's autonomy as progressive:

LEVEL 0: OBSERVE - Brain watches and learns - No actions taken - Purely passive

LEVEL 1: ASSIST - Brain suggests and recommends - Human makes decisions - Brain supports human work

LEVEL 2: EXECUTE WITH HUMAN APPROVAL - Brain proposes actions - Human approves before execution - Explicit approval gate

LEVEL 3: AUTHORIZED AUTOMATION - Brain executes approved workflows autonomously - Only for pre-authorized routine tasks - Human can override at any time

Organization Control

The organization controls the allowed level.

The Brain must not independently increase its own authority.

Permissions are external governance controls.

Autonomy Progression by Ring

RING 0-2: OBSERVE

RING 3-5: ASSIST

RING 6-8: EXECUTE WITH APPROVAL

RING 9+: AUTHORIZED AUTOMATION (future)

V1 Target: Primarily OBSERVE + ASSIST

GOVERNANCE

Governance Hierarchy

The organization decides who is authorized to validate Brain knowledge.

CEO / General Manager

↓

Authorized Knowledge Reviewer

↓

Brain

The exact role may vary by organization.

Configurable Permissions

The system must support configurable permissions for:

- Who can observe?
- Who can review?

- Who can validate?
- Who can edit?
- Who can approve actions?
- Who can change autonomy?
- Who can access sensitive cases?

Auditability

Every important governance decision should be auditable.

PERMISSIONS

Permission Model

Organization-Controlled Authority

- CEO / General Manager sets validation authority
- Roles define permission boundaries
- Access is scoped by organization, domain, case

Validation Authority

- Designated by organization
- Can be role-based or individual
- Can be domain-specific
- Audit trail for all validations

Server-Side Enforcement

Permissions must be enforced server-side.

Frontend permissions are presentation controls, not security controls.

Permission-Aware Retrieval

AI cannot retrieve information the current user is not authorized to access.

All retrieval must respect user permissions.

DATA ARCHITECTURE

Data Categories

Separate and define ownership for each:

Operational Data - CRM data - Case data - Client data - User data - Ownership: Organization

Knowledge Data - Validated knowledge - Knowledge candidates - Domain knowledge - Ownership: Organization (with provenance)

Memory - Embeddings - Vector representations - Ownership: System (derived from knowledge)

Conversations - WhatsApp messages - Email threads - Phone call logs - Ownership: Organization

Cases - Case metadata - Case-specific knowledge - Ownership: Organization

Documents - Original files - Parsed content - Ownership: Organization

Embeddings - Vector representations - Ownership: System (derived)

Metadata - Knowledge object metadata - Provenance data - Ownership: System

Events - System events - Audit logs - Ownership: System

Evaluation Data - Training data - Unseen evaluation data - Ownership: System (isolated from production)

Data Lifecycle

Define lifecycle for each data category:

- Creation
 - Storage
 - Access
 - Update
 - Archive
 - Delete
 - Retention policy
-

AI ARCHITECTURE

AI Component Specification

Every AI capability must specify:

- **Input** - What it consumes
- **Processing** - How it processes
- **Model** - What model is used
- **Output** - What it produces
- **Confidence** - Confidence/uncertainty
- **Evaluation** - How it's evaluated
- **Fallback** - What happens on failure
- **Human Oversight** - Human involvement
- **Logging** - What is logged
- **Failure Modes** - Known failure scenarios

No AI Without Evaluation

No AI feature should be accepted merely because it produces impressive responses.

It must have an evaluation strategy.

AI Engineering vs Data Science

AI Engineering: - LLM integration - Prompt engineering - RAG implementation - Agent orchestration - Context management

Data Science: - Model training - Evaluation - Metrics - Statistical analysis - ML experiments

EVALUATION STRATEGY

Evaluation Dataset Principle

Define separate:

Training / Development Data - Used for development and tuning - Can be seen during development

Unseen Evaluation Data - Used for final validation - Must NOT be used as training data - Represents real-world scenarios

Legal V1 Evaluation

Create a separate unseen evaluation corpus for Legal:

- Test the Brain on cases it has not been explicitly trained on
- Test on legal concepts not in training data
- Measure performance on unseen scenarios

Evaluation Metrics

Retrieval: - Precision@k - Recall@k - MRR (Mean Reciprocal Rank) - NDCG (Normalized Discounted Cumulative Gain)

Classification: - Precision - Recall - F1 Score

Document Understanding: - Extraction accuracy - Clause detection accuracy

Similarity: - Similarity score calibration - Human agreement rate

End-to-End: - Lawyer satisfaction - Time saved - Task completion rate

SOFTWARE ANALYSIS REQUIREMENTS

Required Analysis Documents

The project Constitution explicitly requires the team to produce:

Before Implementation:

1. Project Scope
2. Requirements Specification
3. Stakeholder Analysis
4. Actors
5. Use Cases
6. Functional Requirements
7. Non-Functional Requirements
8. System Context Diagram
9. System Architecture
10. Component Diagram
11. Sequence Diagrams
12. Activity Diagrams
13. Class / Domain Model
14. ERD (Entity Relationship Diagram)
15. Data Flow Diagrams (where useful)

16. State Diagrams (where useful)
17. Deployment Architecture
18. API Contracts
19. Permission Model
20. AI Architecture
21. Data Architecture
22. Integration Architecture

Analysis Before Implementation

Do not allow implementation to replace system analysis.

Analysis must be completed and approved before implementation begins.

UI/UX PRINCIPLES

UI/UX Derived from Analysis

UI/UX must be derived from:

- Actors
- Use Cases
- Workflows
- Permissions
- System States
- Data Models

—not invented independently.

Workflow Understanding

For every major workflow:

User
-> UI
-> API
-> Backend
-> Data / AI
-> Response
-> UI

must be understood before implementation.

Permission-Aware UI

UI must respect permissions: - Hide actions user cannot perform - Show only accessible data - Display permission errors clearly

SECURITY

Enterprise-Grade Baseline Principles

Isolation: - Tenant isolation - Organization isolation - Case isolation

Access Control: - Role-based access - Least privilege - Permission-aware retrieval

Audit: - Audit logs - Sensitive data handling logs - Permission change logs

Data Protection: - Encryption at rest - Encryption in transit - Secret management - Data retention policies

AI Security: - AI access boundaries - Permission-aware AI retrieval - AI output auditing

Critical Security Requirements

AI cannot retrieve information the current user is not authorized to access.

This must be enforced at the server level for all AI operations.

TESTING

Testing Levels

Define testing at multiple levels:

Unit Tests - Individual functions - Service methods - Engine components

Integration Tests - Service interactions - Database operations - API endpoints

End-to-End Tests - Complete user flows - Cross-system integration - Real-world scenarios

System Tests - Full system behavior - Cross-component integration

Security Tests - Permission enforcement - Access control - Case isolation - Data leakage

Data Tests - Data validation - Data quality - Data migration

AI Evaluation - Retrieval quality - Classification accuracy - Reasoning quality - Assistance quality

Human-in-the-Loop Tests - Validation workflows - Governance flows - Permission flows

Regression Tests - Prevent breaking changes - Ensure stability across Rings

Performance Tests (where relevant) - Response time - Throughput - Scalability

Test Categories

Golden Test Cases - Core happy path scenarios - Must always pass

Unseen Evaluation Cases - Separate from training - Used for final validation

Failure Scenarios - System failures - Network failures - AI failures

Conflict Scenarios - Knowledge conflicts - Permission conflicts - Case conflicts

Permission Scenarios - Unauthorized access attempts - Permission escalation attempts - Cross-case access attempts

Multi-Case Isolation Scenarios - Knowledge leakage tests - Case contamination tests - Client separation tests

DOCUMENTATION GOVERNANCE

Documentation Hierarchy

LEVEL 0: PROJECT CONSTITUTION - This document - Highest authority

LEVEL 1: ARCHITECTURE DOCUMENTS - System architecture - Data architecture - AI architecture - Integration architecture

LEVEL 2: RING SPECIFICATIONS - Ring definitions - Ring entry/exit criteria - Ring deliverables

LEVEL 3: DOMAIN SPECIFICATIONS - Domain knowledge - Domain workflows - Domain-specific engines

LEVEL 4: API / DATA / AI SPECIFICATIONS - API contracts - Data schemas - AI model specifications

LEVEL 5: IMPLEMENTATION DOCUMENTATION - Code documentation - API documentation - Deployment guides

Conflict Resolution

Lower-level documents must not contradict higher-level documents.

If a conflict exists:

Higher-level authority wins unless formally amended.

Amendment Process

To amend a higher-level document:

1. Document the proposed change
2. Explain why the change is necessary
3. Get approval from architectural authority
4. Update the document
5. Communicate the change to the team

CHANGE MANAGEMENT

Architecture Change Process

No developer should silently change:

- Core architecture
- Shared schemas
- General Brain interfaces
- Cross-Ring contracts
- Permission model
- Knowledge model
- Domain boundaries

without documenting the change.

Architecture Decision Records (ADR)

Create Architecture Decision Records (ADR) for important architectural decisions.

Each ADR should contain:

- **Context** - What is the situation?
- **Decision** - What was decided?
- **Alternatives** - What alternatives were considered?
- **Reason** - Why was this decision made?
- **Consequences** - What are the consequences?
- **Date** - When was the decision made?
- **Owner** - Who owns this decision?

ADR Template

ADR-XXX: [Title]

Context
[Describe the context]

Decision
[Describe the decision]

Alternatives
[List alternatives considered]

Reason
[Explain the reasoning]

Consequences
[Describe positive and negative consequences]

Date
[Date of decision]

Owner
[Person or team responsible]

RING ARCHITECTURE

Ring Philosophy

Each Ring is a **complete, working vertical capability**.

NOT: - Frontend phase - Backend phase - AI phase - Data Science phase - Then integration

YES: - Frontend + Backend + AI + Data Science + Database + Integration + Testing - Working together where relevant - Complete end-to-end capability

Ring Completion Criteria

A Ring is complete only when: - Implemented - Integrated - Tested - Demonstrable - Architecturally aligned

Ring Classification

Every capability in every Ring must be labeled:

- [GB] GENERAL BRAIN
- [DOMAIN] DOMAIN-SPECIFIC
- [ORG] ORGANIZATION-SPECIFIC
- [SHARED] CROSS-LAYER

This classification is critical for ensuring we build a reusable platform.

CROSS-FUNCTIONAL COLLABORATION

Team Collaboration Model

For every Ring, document:

FRONTEND

What UI/UX is required? - Screens and components - User interactions - Navigation flows

What user interactions exist? - Input forms - Action buttons - Real-time updates

What states exist? - Loading states - Error states - Success states - Empty states

What permissions affect UI? - Hide/show based on permissions - Enable/disable based on permissions

What APIs are required? - API endpoints needed - Data contracts - Error handling

What real-time behavior exists? - WebSocket connections - Live updates - Notifications

BACKEND

What domain logic exists? - Business rules - Validation logic - Processing workflows

What entities exist? - Data models - Relationships - Constraints

What APIs exist? - REST endpoints - GraphQL (if used) - WebSocket events

What events exist? - Domain events - Integration events - System events

What permissions exist? - Permission checks - Role-based access - Resource ownership

What integrations exist? - External systems - Third-party APIs - Internal services

AI / ML

What intelligence capability exists? - What AI task is being performed? - What is the expected behavior?

What inputs does it consume? - Data sources - Context - Parameters

What outputs does it produce? - Response format - Confidence scores - Citations

What models are used? - LLM models - Custom models - Ensemble approaches

What evaluation is required? - Metrics - Test cases - Baseline comparisons

What confidence / uncertainty exists? - How is confidence calculated? - How is uncertainty expressed?

What human validation exists? - When do humans review? - How is feedback incorporated?

DATA SCIENCE

What data is required? - Training data - Evaluation data - Production data

How is it collected? - Data sources - Collection methods - Data pipelines

How is it cleaned? - Preprocessing steps - Normalization - Validation

How is it structured? - Schema definition - Feature engineering - Representation

What features / representations are needed? - Embeddings - Vectors - Graph structures

What evaluation datasets exist? - Training set - Validation set - Unseen test set

What metrics are required? - Performance metrics - Quality metrics - Business metrics

DATABASE

What persistent entities exist? - Tables - Collections - Graph nodes

What relationships exist? - Foreign keys - Graph edges - Indexes

What is transactional? - ACID requirements - Transaction boundaries

What is knowledge? - Knowledge objects - Provenance data - Validation history

What is memory? - Embeddings - Vector representations - Cache

What requires vector / semantic retrieval? - Text search - Similarity search - Semantic matching

What requires graph / relational representation? - Relationships - Hierarchies - Provenance chains

INTEGRATION

What external systems are connected? - WhatsApp - CRM - Email - Document systems

What events flow in? - Webhooks - Polling - Real-time streams

What events flow out? - Notifications - Updates - Actions

What permissions are required? - API keys - OAuth tokens - Access scopes

What failure modes exist? - Network failures - API rate limits - Service downtime

TESTING

What unit tests exist? - Function-level tests - Service-level tests

What integration tests exist? - API tests - Database tests - Integration tests

What AI evaluation exists? - Retrieval evaluation - Classification evaluation - Reasoning evaluation

What end-to-end scenarios exist? - User workflows - Cross-system flows

What acceptance criteria exist? - Must-have features - Quality thresholds - Performance requirements

RING ENTRY/EXIT CRITERIA

For Every Ring

ENTRY CRITERIA

What must exist before work begins?

- Previous Ring complete
- Dependencies satisfied
- Requirements defined
- Architecture approved
- Team allocated
- Environment ready

DELIVERABLES

What must be produced?

- Frontend components
- Backend services
- AI models/logic
- Data pipelines
- Database schemas
- Integration code
- Test suites
- Documentation

INTEGRATION CONTRACTS

What must connect to other Rings?

- API contracts
- Data schemas
- Event contracts
- Permission contracts

TESTING

What must be tested?

- Unit tests
- Integration tests
- AI evaluation
- End-to-end tests
- Security tests
- Performance tests

EXIT CRITERIA

When can the Ring officially be marked complete?

- All deliverables complete
- All tests passing
- Integration verified
- Documentation complete
- Demo ready
- Architectural review passed

DEMO

What must be demonstrable?

- Working end-to-end flow
- Key features demonstrated

- Quality metrics met
- Stakeholder approval

DOCUMENTATION

What must be documented?

- Architecture decisions
 - API documentation
 - Data schemas
 - Test results
 - User guides
 - Deployment guides
-

V1 DEFINITION

V1 Boundary

V1 must demonstrate the product vision without pretending to be a production-scale enterprise platform.

V1 Must-Have

At minimum, the V1 should demonstrate:

- General Brain
- Legal Domain
- Organization Context
- Knowledge Acquisition
- Human Validation
- Persistent Memory
- Retrieval
- Reasoning
- Conflict Detection
- Case Understanding
- WhatsApp / communication integration
- CRM / operational integration
- Decision Support
- Limited AI Employee
- Learning Feedback Loop

V1 Should-Have

- Insights dashboard
- Governance dashboard
- Activity monitoring
- Multiple test organizations
- Performance optimization

V1 Future (Post-V1)

- Multiple domains
- Advanced autonomy
- Full CRM replacement
- Enterprise-scale deployment
- Advanced analytics

Scope Control

Do not allow scope creep.

V1 is about proving the architecture, not building a production enterprise platform.

FUTURE DOMAIN STRATEGY

Legal as Proof

The Legal implementation must be treated as the first proof that the architecture can generalize.

Adding a New Domain

After Legal:

Sales, HR, Finance, Customer Support, etc. should reuse the General Brain.

Domain Addition Process

For every future Domain Module, define:

- Domain Knowledge
- Domain Workflows
- Domain Tools
- Domain-specific Retrieval
- Domain-specific Reasoning (where needed)
- Domain UI
- Domain Integrations

without rebuilding the Brain.

Example

Existing:

General Brain
+
Legal Domain
+
Organization A

New:

General Brain (reused)
+
Sales Domain (new)
+
Organization B (new)

The same General Brain capabilities remain reusable.

TEAM OPERATING MODEL

Feature Ownership

Every feature should have:

- **Owner** - Person or team responsible
- **Dependencies** - What it depends on
- **Inputs** - What it needs
- **Outputs** - What it produces
- **Acceptance Criteria** - Definition of done
- **Test Plan** - How it will be tested
- **Documentation** - What will be documented

Cross-Team Contracts

Cross-team contracts must be explicit.

- Frontend should not guess APIs
- Backend should not guess UI requirements
- AI should not guess data schemas
- Data Science should not guess evaluation requirements

Shared Contracts

Everyone works from shared contracts:

- API contracts
 - Data schemas
 - Event contracts
 - Permission models
 - Test plans
-

SOURCE OF TRUTH HIERARCHY

Hierarchy

1. **Project Constitution** (this document)
2. **Approved Architecture Decisions** (ADRs)
3. **Ring Specifications**
4. **Domain Specifications**
5. **API / Data / AI Contracts**
6. **Implementation**
7. **Temporary notes**

Conflict Resolution

If implementation contradicts architecture:

Architecture must be reviewed.

If a requirement contradicts the Constitution:

The Constitution must be formally amended.

Never Silently Drift

Never silently drift from the Constitution.

All architectural deviations must be documented and approved.

MASTER ROADMAP

Phase-Based Roadmap

PHASE 0: Project Scope & Constitution
V
PHASE 1: Software Requirements Analysis
V
PHASE 2: System Analysis & Modeling
V
PHASE 3: General Brain Architecture (Rings 0-2)
V
PHASE 4: Legal Domain Intelligence (Ring 3)
V
PHASE 5: Organization Intelligence (Ring 4)
V
PHASE 6: Case Intelligence (Ring 5)
V
PHASE 7: Communication + CRM Integrations (Ring 6)
V
PHASE 8: Knowledge Learning Loop (Ring 8)
V
PHASE 9: Decision Support + AI Employee (Ring 7)
V
PHASE 10: Insights & Governance (Ring 9)
V
PHASE 11: Legal V1 Integration & Evaluation (Ring 10)
V
PHASE 12: Generalization & Next Domains (Ring 11+)

Ring Dependency Graph

RING 0: FOUNDATION
V
RING 1: BRAIN CORE
V
RING 2: KNOWLEDGE & MEMORY
V
RING 3: LEGAL DOMAIN
V
RING 4: ORGANIZATION CONTEXT
V
RING 5: CASE INTELLIGENCE
V
RING 6: OBSERVATION (WhatsApp)
V
RING 7: REASONING & ASSISTANCE
V
RING 8: LEARNING & VALIDATION
V
RING 9: INSIGHTS & GOVERNANCE
V
RING 10: INTEGRATED LEGAL V1
V
RING 11+: FUTURE DOMAINS

DEFINITION OF DONE

Universal Definition of Done for Rings

A Ring is NOT DONE because:

- “the frontend works”
- “the API works”
- “the AI responds”

It is DONE only when:

Frontend - Implemented - Tested - Integrated

Backend - Implemented - Tested - Integrated

Data - Schemas defined - Migrations complete - Tested

AI - Implemented - Evaluated - Tested

Database - Schemas deployed - Data seeded - Tested

Integration - External systems connected - Tested - Documented

Permissions - Enforced - Tested - Documented

Testing - Unit tests pass - Integration tests pass - AI evaluation passes - End-to-end tests pass

Documentation - Architecture documented - APIs documented - User guides written

are integrated and verified according to the Ring’s acceptance criteria.

DETAILED RING SPECIFICATIONS

RING 0 — FOUNDATION

Purpose: Establish development environment and core infrastructure

Why Now: Cannot build anything without foundation

Dependencies: None

Capabilities Introduced: - Development environment setup - Database infrastructure - Basic API framework - Frontend framework - AI/LLM infrastructure - Data pipeline infrastructure

Architecture Impact: - Establishes technology stack - Defines project structure - Sets up CI/CD

Frontend Tasks: - Setup React + TypeScript + Vite - Setup Tailwind CSS - Create basic UI skeleton - Setup routing

Backend Tasks: - Setup Node.js/Express or equivalent - Setup database connections - Create API structure - Setup authentication skeleton

AI Tasks: - Setup LLM access (OpenAI or local) - Create AI service skeleton - Setup prompt management

Data Science Tasks: - Setup data pipeline infrastructure - Setup evaluation framework - Setup data storage

Database/Data Tasks: - Setup PostgreSQL - Setup vector database (pgvector or Qdrant) - Setup graph database (Neo4j or V2) - Setup object storage (S3-compatible)

Integration Tasks: - None

Testing Tasks: - Setup testing framework - Setup CI/CD - Create basic unit tests

Deliverables: - Running development environment - All infrastructure services running - Basic project structure - CI/CD pipeline

Definition of Done: - All team members can run the project locally - All infrastructure services are accessible - Basic API endpoints respond - Basic UI renders - CI/CD passes

Risks: - Infrastructure complexity - Team unfamiliarity with stack

What the Next Ring Unlocks: - Core Brain infrastructure can be built

IMPLEMENTATION GUIDANCE & TOOLS

Frontend Implementation: - **Tools:** React 18, TypeScript, Vite, Tailwind CSS v4, React Router - **Setup Commands:** `bash npm create vite@latest organizational-brain -- --template react-ts cd organizational-brain npm install npm install -D tailwindcss postcss autoprefixer npm install react-router-dom npx tailwindcss init -p` - **Project Structure:** `src/ components/ # Reusable components pages/ # Page components hooks/ # Custom React hooks services/ # API service calls types/ # TypeScript type definitions utils/ # Utility functions config/ # Configuration files` - **Configuration:** Configure Tailwind in `tailwind.config.js`, set up routing in `App.tsx`

Backend Implementation: - **Tools:** Node.js, Express, TypeScript, Prisma (ORM), JWT (authentication) - **Setup Commands:** `bash mkdir backend && cd backend npm init -y npm install express cors dotenv helmet morgan npm install -D typescript @types/node @types/express ts-node nodemon npm install prisma @prisma/client npm install jsonwebtoken bcryptjs npm install -D @types/jsonwebtoken @types/bcryptjs npx tsc --init npx prisma init` - **Project Structure:** `src/ controllers/ # Request handlers services/ # Business logic models/ # Prisma models middleware/ # Express middleware routes/ # API route definitions utils/ # Utility functions config/ # Configuration` - **Database Setup:** Configure PostgreSQL connection in `.env`, run Prisma migrations

AI/LLM Implementation: - **Tools:** OpenAI API (or Ollama for local), LangChain (or custom orchestration) - **Setup Commands:** `bash npm install openai npm install langchain` - **Configuration:** Store API keys in `.env` file, create AI service wrapper - **Prompt Management:** Create prompts/ directory with prompt templates

Data Science Implementation: - **Tools:** Python, Jupyter, Pandas, NumPy, Scikit-learn - **Setup Commands:** `bash python -m venv venv venv source venv/bin/activate pip install jupyter pandas numpy scikit-learn matplotlib seaborn` - **Project Structure:** `data-science/ notebooks/ # Jupyter notebooks scripts/ # Python scripts data/ # Raw and processed data models/ # Saved models evaluations/ # Evaluation results`

Database Setup: - **PostgreSQL:** Install via Docker or native, create database `organizational_brain` - **Vector Database:** Install pgvector extension for PostgreSQL or Qdrant `bash # For pgvector: docker run --name pgvector -e POSTGRES_PASSWORD=password -p 5432:5432 -d pgvector/pgvector # For Qdrant: docker run -p 6333:6333 -p 6334:6334 -d qdrant/qdrant` - **Graph Database:** Install Neo4j via Docker `bash docker run -p 7474:7474 -p 7687:7687 -e NEO4J_AUTH=neo4j/password -d neo4j` - **Object Storage:** Use MinIO (S3-compatible) locally or AWS S3 `bash docker run -p 9000:9000 -p 9001:9001 -e MINIO_ROOT_USER=minioadmin -e MINIO_ROOT_PASSWORD=minioadmin -d minio/minio`

Testing Setup: - **Frontend:** Vitest (comes with Vite), React Testing Library `bash npm install -D vitest @testing-library/react @testing-library/jest-dom` - **Backend:** Jest, Supertest `bash npm install -D jest @types/jest supertest @types/supertest ts-jest` - **CI/CD:** GitHub Actions or GitLab CI - Create `.github/workflows/ci.yml` for automated testing - Configure automated deployment on merge to main

RING 1 — BRAIN CORE

Purpose: Implement core General Brain engines

Why Now: Foundation is ready, need core intelligence infrastructure

Dependencies: Ring 0

Capabilities Introduced: - Perception Engine [GB] - Observation Engine [GB] - Context Engine [GB] - Basic Memory Engine [GB] - Basic Retrieval Engine [GB]

Architecture Impact: - Establishes General Brain engine interfaces - Defines observation pipeline - Establishes context model

Frontend Tasks: - Brain dashboard (status, activity) - Observation viewer - Context switcher

Backend Tasks: - Perception service - Observation service - Context service - Memory service (basic) - Retrieval service (basic)

AI Tasks: - Perception implementation - Context resolution - Basic retrieval logic

Data Science Tasks: - Define observation schema - Define context schema - Define knowledge object schema

Database/Data Tasks: - Create observation tables - Create context tables - Create knowledge tables - Setup vector indexes

Integration Tasks: - None

Testing Tasks: - Engine unit tests - Integration tests - Context isolation tests

Deliverables: - Running Brain core engines - Observation pipeline working - Context resolution working - Basic memory storage working - Basic retrieval working

Definition of Done: - Can observe events - Can resolve context (organization, domain, case) - Can store observations - Can retrieve stored observations - Context isolation verified

Risks: - Overengineering core engines - Premature optimization

What the Next Ring Unlocks: - Knowledge and Memory infrastructure

IMPLEMENTATION GUIDANCE & TOOLS

Frontend Implementation: - **Tools:** React, TypeScript, Recharts (for dashboards), Socket.io-client (for real-time) - **Setup:** `bash npm install recharts socket.io-client npm install lucide-react -`

Components to Create: - `BrainDashboard.tsx` - Main dashboard showing engine status - `ObservationViewer.tsx` - Real-time observation stream display - `ContextSwitcher.tsx` - Dropdown to switch between org/domain/case context - **State Management:** Use React Context for global state (`BrainContext`, `ContextContext`) - **Real-time Updates:** Connect via WebSocket to receive observation updates

Backend Implementation: - **Tools:** Express, Bull (job queue), Socket.io (real-time), Prisma - **Setup:** `bash npm install bull socket.io npm install -D @types/bull @types/socket.io` - **Services to Create:** - `PerceptionService.ts` - Handles raw input processing - `ObservationService.ts` - Manages observation stream and buffering - `ContextService.ts` - Resolves and manages contextual boundaries - `MemoryService.ts` - Basic memory storage operations - `RetrievalService.ts` - Basic retrieval logic - **API Endpoints:** - `GET /api/brain/status` - Engine status - `GET /api/observations` - List observations - `POST /api/context/resolve` - Resolve context - `GET /api/memory/:id` - Retrieve from memory - **Real-time:** Use Socket.io to push observations to frontend in real-time

AI Implementation: - **Tools:** OpenAI API, LangChain - **Services to Create:** - `PerceptionAI.ts` - LLM-based perception (text classification, entity extraction) - `ContextAI.ts` - Context resolution using LLM - `RetrievalAI.ts` - Basic semantic retrieval - **Prompt Templates:** Create prompts for perception and context resolution - **Configuration:** Configure model selection (GPT-4 for complex tasks, GPT-3.5 for simple)

Data Science Implementation: - **Tools:** Python, Pandas, Scikit-learn - **Tasks:** - Define observation schema in Prisma schema - Define context schema (organization, domain, case) - Define knowledge object schema - **Jupyter Notebooks:** - 01_observation_schema.ipynb - Design observation data structure - 02_context_model.ipynb - Design context resolution logic - 03_knowledge_object.ipynb - Design knowledge object structure

Database Implementation: - **Prisma Schema:** `prisma model Observation { id String @id @default(uuid()) type String source String content Json timestamp DateTime @default(now()) contextId String? context Context? @relation(fields: [contextId], references: [id]) }`

`model Context { id String @id @default(uuid()) organizationId String domainId String? caseId String? userId String? metadata Json observations Observation[] }`

`model KnowledgeObject { id String @id @default(uuid()) type String content Json confidence Float timestamp DateTime @default(now()) }` - **Vector Setup:** Create vector table with pgvector for embeddings - **Indexes:** Create indexes on observation type, timestamp, contextId

Testing Implementation: - **Unit Tests:** - Test each service independently - Mock external dependencies (LLM, database) - **Integration Tests:** - Test observation pipeline end-to-end - Test context resolution with real data - **Context Isolation Tests:** - Verify observations from one context don't leak to another

RING 2 — KNOWLEDGE & MEMORY

Purpose: Implement knowledge extraction, storage, and retrieval

Why Now: Brain core is ready, need knowledge infrastructure

Dependencies: Ring 1

Capabilities Introduced: - Knowledge Engine [GB] - Memory / Storage Engine [GB] - Retrieval Engine [GB] - Provenance Engine [GB]

Architecture Impact: - Establishes knowledge object model - Implements multi-store architecture - Establishes provenance tracking

Frontend Tasks: - Knowledge viewer - Knowledge editor (basic) - Provenance viewer

Backend Tasks: - Knowledge service - Multi-store coordination - Provenance service - Advanced retrieval

AI Tasks: - Knowledge extraction - Embedding generation - Retrieval algorithms - RAG implementation

Data Science Tasks: - Define knowledge taxonomy - Define embedding strategy - Create retrieval evaluation dataset - Define retrieval metrics

Database/Data Tasks: - Implement multi-store architecture - Setup vector embeddings - Setup graph relationships - Implement provenance tracking

Integration Tasks: - None

Testing Tasks: - Knowledge extraction tests - Retrieval evaluation - Provenance tracking tests - Multi-store coordination tests

Deliverables: - Knowledge extraction working - Multi-store storage working - Retrieval working with metrics - Provenance tracking working

Definition of Done: - Can extract knowledge from documents - Can store in multi-store architecture - Can retrieve with measurable quality - Provenance is tracked for all knowledge - Retrieval evaluation meets baseline metrics

Risks: - Poor retrieval quality - Complex multi-store coordination - Provenance complexity

What the Next Ring Unlocks: - Domain-specific knowledge can be loaded

IMPLEMENTATION GUIDANCE & TOOLS

Frontend Implementation: - **Tools:** React, TypeScript, Monaco Editor (for knowledge editing), React Query (for data fetching) - **Setup:** `bash npm install @monaco-editor/react @tanstack/react-query` - **Components to Create:** - `KnowledgeViewer.tsx` - Display knowledge objects with filters - `KnowledgeEditor.tsx` - Basic editor for knowledge content - `ProvenanceViewer.tsx` - Show knowledge source and validation history - **Data Fetching:** Use React Query for caching and optimistic updates - **Editor:** Use Monaco Editor for syntax highlighting in knowledge content

Backend Implementation: - **Tools:** Express, Prisma, pgvector (for vector operations), Redis (caching) - **Setup:** `bash npm install pgvector redis npm install -D @types/pgvector @types/redis` - **Services to Create:** - `KnowledgeService.ts` - Knowledge extraction and management - `MultiStoreService.ts` - Coordinate between relational, vector, graph stores - `ProvenanceService.ts` - Track source and validation history - `AdvancedRetrievalService.ts` - Hybrid retrieval (keyword + semantic) - **API Endpoints:** - `POST /api/knowledge/extract` - Extract knowledge from documents - `GET /api/knowledge/:id` - Retrieve knowledge object - `GET /api/knowledge/search` - Search knowledge - `GET /api/knowledge/provenance/:id` - Get provenance chain - **Multi-Store Coordination:** - Relational: Store structured knowledge metadata - Vector: Store embeddings for semantic search - Graph: Store relationships between knowledge objects

AI Implementation: - **Tools:** OpenAI API, LangChain, tiktoken (token counting) - **Setup:** `bash npm install tiktoken` - **Services to Create:** - `KnowledgeExtractionAI.ts` - Extract knowledge from documents using LLM - `EmbeddingAI.ts` - Generate embeddings using OpenAI text-embedding-3 - `RetrievalAI.ts` - RAG implementation with LangChain - `RAGOrchestrator.ts` - Coordinate retrieval and generation - **Embedding Strategy:** - Use OpenAI text-embedding-3-small for cost efficiency - Batch embeddings for efficiency - Cache embeddings in Redis - **RAG Implementation:** - Use LangChain's RetrievalQA chain - Configure chunking strategy (e.g., 500 tokens with 50 overlap) - Implement hybrid retrieval (BM25 + semantic)

Data Science Implementation: - **Tools:** Python, Pandas, Scikit-learn, FAISS (vector similarity) - **Setup:** `bash pip install faiss-cpu sentence-transformers` - **Tasks:** - Define knowledge taxonomy (hierarchy of knowledge types) - Define embedding strategy (model selection, chunking parameters) - Create retrieval evaluation dataset - Define retrieval metrics (Precision@k, Recall@k, MRR, NDCG) - **Jupyter Notebooks:** - `01_knowledge_taxonomy.ipynb` - Design knowledge classification system - `02_embedding_strategy.ipynb` - Test different embedding models - `03_retrieval_evaluation.ipynb` - Evaluate retrieval quality - `04_chunking_experiments.ipynb` - Optimize document chunking

Database Implementation: - **Prisma Schema Extensions:** ```prisma model KnowledgeObject { id String @id @default(uuid()) type String // fact, rule, procedure, pattern content Json embedding Unsupported("vector(1536)")? confidence Float status String // candidate, validated, rejected source String provenance Json timestamp DateTime @default(now()) layer String // general, domain, organization, case domainId String? orgId String? caseId String? }`

`model Provenance { id String @id @default(uuid()) knowledgeId String source String sourceType String // document, conversation, human validatorId String? validated DateTime? metadata Json timestamp DateTime @default(now()) }` - **Vector Database:** - Create vector extension in PostgreSQL: `CREATE EXTENSION vector;` - Create vector index: `CREATE INDEX ON knowledgeobjects USING ivfflat (embedding vector_cosine_ops);` - **Graph Database:** - Create nodes for knowledge objects - Create relationships (RELATED_TO, DERIVED_FROM, CONTRADICTS) - **Caching:** - Use Redis for caching frequent retrievals - Cache embeddings to reduce API calls

Testing Implementation: - **Unit Tests:** - Test knowledge extraction with sample documents - Test embedding generation - Test retrieval algorithms - **Retrieval Evaluation:** - Create evaluation dataset with known relevant documents - Measure Precision@5, Recall@10, MRR - Compare different retrieval strategies

- **Provenance Tests:** - Verify provenance chain is complete - Test provenance tracking across knowledge updates - **Multi-Store Tests:** - Test coordination between stores - Verify data consistency

RING 3 — LEGAL DOMAIN

Purpose: Load Legal domain knowledge and implement domain-specific engines

Why Now: Knowledge infrastructure is ready, need first domain

Dependencies: Ring 2

Capabilities Introduced: - Legal Domain Knowledge [DOMAIN] - Legal Retrieval [DOMAIN] - Legal Classification [DOMAIN] - Legal Document Understanding [DOMAIN]

Architecture Impact: - First domain implementation - Establishes domain adapter pattern - Validates General Brain reusability

Frontend Tasks: - Legal domain UI - Legal document viewer - Legal search interface

Backend Tasks: - Legal domain service - Legal retrieval adapter - Legal classification service - Legal document understanding service

AI Tasks: - Legal retrieval algorithms - Legal classification models - Legal document understanding - Legal reasoning patterns

Data Science Tasks: - Acquire Egyptian Civil Law corpus - Clean and normalize legal documents - Create legal taxonomy - Create legal classification dataset - Create legal retrieval evaluation set - Define legal-specific metrics

Database/Data Tasks: - Load legal corpus into knowledge stores - Create legal-specific indexes - Create legal taxonomy tables

Integration Tasks: - None

Testing Tasks: - Legal retrieval evaluation - Legal classification evaluation - Legal document understanding tests - Domain isolation tests

Deliverables: - Legal corpus loaded - Legal retrieval working - Legal classification working - Legal document understanding working - Domain evaluation metrics met

Definition of Done: - Legal domain knowledge is available - Can retrieve legal knowledge with domain-specific quality - Can classify legal documents - Can understand legal documents - Legal evaluation meets baseline - Domain is isolated from General Brain

Risks: - Legal corpus quality - Domain leakage into General Brain - Legal-specific complexity

What the Next Ring Unlocks: - Organization context can be configured

IMPLEMENTATION GUIDANCE & TOOLS

Frontend Implementation: - **Tools:** React, TypeScript, React PDF (for document viewing), React Select - **Setup:** `bash npm install @react-pdf/renderer react-select` - **Components to Create:** - `LegalDomainUI.tsx` - Legal-specific interface - `LegalDocumentViewer.tsx` - View legal documents with highlighting - `LegalSearchInterface.tsx` - Legal-specific search with filters - **Legal UI Features:** - Document type filters (laws, regulations, precedents) - Legal concept highlighting - Citation linking

Backend Implementation: - **Tools:** Express, Prisma, PDF parsing libraries - **Setup:** `bash npm install pdf-parse mammoth npm install -D @types/pdf-parse` - **Services to Create:** - `LegalDomainService.ts`

- Legal domain-specific operations - LegalRetrievalAdapter.ts - Legal-specific retrieval logic - LegalClassificationService.ts - Document type classification - LegalDocumentUnderstandingService.ts - Clause extraction - **API Endpoints:** - POST /api/legal/upload - Upload legal documents - GET /api/legal/search - Legal-specific search - POST /api/legal/classify - Classify legal documents - POST /api/legal/understand - Extract legal clauses - **Document Processing:** - Use pdf-parse for PDF extraction - Use mammoth for Word document extraction - Implement text cleaning and normalization

AI Implementation: - **Tools:** OpenAI API, LangChain, specialized legal prompts - **Services to Create:** - LegalRetrievalAI.ts - Legal-specific retrieval with domain weighting - LegalClassificationAI.ts - Classify legal document types - LegalDocumentUnderstandingAI.ts - Extract legal clauses and provisions - LegalReasoningAI.ts - Legal reasoning patterns - **Prompt Templates:** - Create legal-specific prompts for classification - Create clause extraction prompts - Create legal reasoning prompts - **Domain Adaptation:** - Use domain-specific prompts - Implement legal terminology recognition - Configure legal citation patterns

Data Science Implementation: - **Tools:** Python, Pandas, Scikit-learn, spaCy (NLP) - **Setup:** bash pip install spacy arabic_reshaper python -m spacy download en_core_web_sm - **Tasks:** - Acquire Egyptian Civil Law corpus - Clean and normalize legal documents - Create legal taxonomy (laws, regulations, precedents) - Create legal classification dataset - Create legal retrieval evaluation set - Define legal-specific metrics - **Jupyter Notebooks:** - 01_legal_corpus_acquisition.ipynb - Scrape and collect legal documents - 02_legal_normalization.ipynb - Clean and normalize legal text - 03_legal_taxonomy.ipynb - Design legal classification system - 04_legal_classification_dataset.ipynb - Create labeled dataset - 05_legal_retrieval_eval.ipynb - Evaluate legal retrieval quality - **Legal Corpus:** - Egyptian Civil Law articles - Court precedents - Legal regulations - Legal commentaries

Database Implementation: - **Prisma Schema Extensions:** ```prisma model LegalDocument { id String @id @default(uuid()) type String // law, regulation, precedent title String content Text source String year Int? articleNum String? embedding Unsupported("vector(1536)")? metadata Json timestamp DateTime @default(now()) }

model LegalConcept { id String @id @default(uuid()) name String category String definition String relatedDocs String[] } ``` - **Legal Indexes:** - Create indexes on document type, year, article number - Create vector index for semantic legal search - **Domain Isolation:** - Store legal knowledge with domainId = 'legal' - Ensure legal-specific data doesn't leak to general brain

Testing Implementation: - **Legal Retrieval Evaluation:** - Create legal-specific test queries - Measure relevance of legal results - Compare with baseline retrieval - **Legal Classification Tests:** - Test classification of various legal document types - Measure precision, recall, F1 - **Legal Document Understanding Tests:** - Test clause extraction accuracy - Test legal concept recognition - **Domain Isolation Tests:** - Verify legal data is isolated from general brain - Test cross-domain contamination

RING 4 — ORGANIZATION CONTEXT

Purpose: Implement organization configuration, roles, and permissions

Why Now: Domain is ready, need organization layer

Dependencies: Ring 3

Capabilities Introduced: - Organization Configuration [ORG] - Roles & Permissions [ORG] - Validation Authority setup [ORG] - Organization Knowledge base [ORG]

Architecture Impact: - Establishes organization isolation - Implements permission model - Enables organization-specific behavior

Frontend Tasks: - Organization setup wizard - Role management UI - Permission management UI - Validation authority configuration

Backend Tasks: - Organization service - Role & permission service - Validation authority service - Organization context service

AI Tasks: - Organization context resolution - Permission-aware reasoning - Organization-specific assistance

Data Science Tasks: - Define organization schema - Define role taxonomy - Define permission model

Database/Data Tasks: - Create organization tables - Create role tables - Create permission tables - Implement organization isolation

Integration Tasks: - None

Testing Tasks: - Organization isolation tests - Permission tests - Role-based access tests - Validation authority tests

Deliverables: - Can configure organization - Can define roles and permissions - Can set validation authority - Organization context is isolated - Permissions are enforced

Definition of Done: - Can create and configure organization - Roles and permissions work - Validation authority can be set - Organization knowledge is isolated - Permissions are enforced in all operations

Risks: - Complex permission model - Organization leakage - Permission bugs

What the Next Ring Unlocks: - Case model can be implemented

IMPLEMENTATION GUIDANCE & TOOLS

Frontend Implementation: - **Tools:** React, TypeScript, React Hook Form (for forms), React Select - **Setup:** `bash npm install react-hook-form @hookform/resolvers zod npm install react-select` - **Components to Create:** - `OrganizationSetupWizard.tsx` - Multi-step organization setup - `RoleManagementUI.tsx` - Create and manage roles - `PermissionManagementUI.tsx` - Configure permissions - `ValidationAuthorityConfig.tsx` - Set validation authority - **Form Validation:** Use Zod for schema validation - **Wizard Flow:** Step-by-step organization setup with progress indicator

Backend Implementation: - **Tools:** Express, Prisma, Casbin (access control library) - **Setup:** `bash npm install casbin casbin-sequelize-adapter` - **Services to Create:** - `OrganizationService.ts` - Organization CRUD operations - `RolePermissionService.ts` - Role and permission management - `ValidationAuthorityService.ts` - Validation authority configuration - `OrganizationContextService.ts` - Resolve organization context - **API Endpoints:** - `POST /api/organization/create` - Create organization - `PUT /api/organization/:id` - Update organization - `POST /api/roles/create` - Create role - `POST /api/permissions/assign` - Assign permissions to role - `POST /api/validation-authority/set` - Set validation authority - **Access Control:** - Use Casbin for permission enforcement - Define policy model (RBAC) - Implement permission middleware

AI Implementation: - **Tools:** OpenAI API, LangChain - **Services to Create:** - `OrganizationContextAI.ts` - Resolve organization context - `PermissionAwareReasoningAI.ts` - Reasoning with permission awareness - `OrganizationSpecificAssistanceAI.ts` - Organization-tailored assistance - **Context Resolution:** - Use LLM to understand organization-specific context - Incorporate organization policies into reasoning - Respect permission boundaries in AI responses

Data Science Implementation: - **Tools:** Python, Pandas - **Tasks:** - Define organization schema - Define role taxonomy (admin, lawyer, paralegal, etc.) - Define permission model (resource-action pairs) - **Jupyter Notebooks:** - `01_organization_schema.ipynb` - Design organization data structure - `02_role_taxonomy.ipynb` - Design role hierarchy - `03_permission_model.ipynb` - Design permission matrix

Database Implementation: - **Prisma Schema Extensions:** ```prisma model Organization { id String @id @default(uuid()) name String domain String // legal, sales, hr, etc. config Json // policies, workflows, settings`

createdAt DateTime @default(now()) updatedAt DateTime @updatedAt users User[] roles Role[] cases Case[] }

model Role { id String @id @default(uuid()) name String organizationId String permissions Json // permission list organization Organization @relation(fields: [organizationId], references: [id]) users User[] }

model Permission { id String @id @default(uuid()) resource String // knowledge, case, observation, etc. action String // read, write, validate, delete description String }

model User { id String @id @default(uuid()) email String @unique name String organizationId String roleId String organization Organization @relation(fields: [organizationId], references: [id]) role Role @relation(fields: [roleId], references: [id]) }

model ValidationAuthority { id String @id @default(uuid()) organizationId String @unique authorizedRoles Json // roles that can validate organization Organization @relation(fields: [organizationId], references: [id]) }
`` - **Organization Isolation:** - Add organizationId to all relevant tables - Create indexes on organizationId - Implement row-level security - **Permission Storage:** - Store permissions as JSON in role table - Create permission lookup table for efficiency

Testing Implementation: - **Organization Isolation Tests:** - Verify data from one organization doesn't leak to another - Test cross-organization queries - **Permission Tests:** - Test role-based access control - Test permission enforcement on API endpoints - Test permission escalation attempts - **Role-Based Access Tests:** - Test different roles have appropriate access - Test role inheritance - **Validation Authority Tests:** - Test validation authority configuration - Test only authorized roles can validate

RING 5 — CASE INTELLIGENCE

Purpose: Implement case model, client management, and case isolation

Why Now: Organization context is ready, need case model

Dependencies: Ring 4

Capabilities Introduced: - Case Model [SHARED] - Client Management [SHARED] - Case Isolation [SHARED] - Case Knowledge [ORG]

Architecture Impact: - Establishes case as first-class object - Implements strict case isolation - Enables case-specific intelligence

Frontend Tasks: - Case workspace - Client management UI - Case viewer - Case knowledge viewer

Backend Tasks: - Case service - Client service - Case isolation middleware - Case knowledge service

AI Tasks: - Case context resolution - Case-specific reasoning - Case isolation enforcement

Data Science Tasks: - Define case schema - Define client schema - Define case knowledge schema

Database/Data Tasks: - Create case tables - Create client tables - Implement case isolation - Create case knowledge tables

Integration Tasks: - None

Testing Tasks: - Case isolation tests (critical) - Case knowledge leakage tests - Client management tests - Case context resolution tests

Deliverables: - Case model working - Client management working - Case isolation verified - Case knowledge isolated - Case context resolution working

Definition of Done: - Can create cases and clients - Case knowledge is strictly isolated - No knowledge leaks between cases - Case context is correctly resolved - Case isolation tests pass

Risks: - Case contamination - Complex case relationships - Isolation bugs

What the Next Ring Unlocks: - Observation can be tied to cases

IMPLEMENTATION GUIDANCE & TOOLS

Frontend Implementation: - **Tools:** React, TypeScript, React Query, React Beautiful DnD (for drag-drop) - **Setup:** `bash npm install @tanstack/react-query @dnd-kit/core @dnd-kit/sortable` - **Components to Create:** - `CaseWorkspace.tsx` - Main case workspace interface - `ClientManagementUI.tsx` - Client CRUD operations - `CaseViewer.tsx` - View case details and timeline - `CaseKnowledgeViewer.tsx` - View case-specific knowledge - **Case UI Features:** - Case timeline visualization - Client information panel - Case status tracking - Document organization

Backend Implementation: - **Tools:** Express, Prisma, middleware for isolation - **Services to Create:** - `CaseService.ts` - Case CRUD operations - `ClientService.ts` - Client management - `CaseIsolationMiddleware.ts` - Enforce case isolation - `CaseKnowledgeService.ts` - Case-specific knowledge operations - **API Endpoints:** - `POST /api/cases/create` - Create case - `GET /api/cases/:id` - Get case details - `POST /api/clients/create` - Create client - `GET /api/cases/:id/knowledge` - Get case knowledge - **Case Isolation Middleware:** - Verify user has access to case - Filter queries by `caseId` - Prevent cross-case data access

AI Implementation: - **Tools:** OpenAI API, LangChain - **Services to Create:** - `CaseContextAI.ts` - Resolve case-specific context - `CaseSpecificReasoningAI.ts` - Reason within case boundaries - `CaseIsolationEnforcementAI.ts` - Enforce isolation in AI responses - **Context Resolution:** - Always resolve case context before reasoning - Include case history in context - Respect case boundaries in all AI operations

Data Science Implementation: - **Tools:** Python, Pandas - **Tasks:** - Define case schema - Define client schema - Define case knowledge schema - **Jupyter Notebooks:** - `01_case_schema.ipynb` - Design case data structure - `02_client_schema.ipynb` - Design client data structure - `03_case_knowledge_schema.ipynb` - Design case-specific knowledge structure

Database Implementation: - **Prisma Schema Extensions:** ```prisma model Case { id String @id @default(uuid()) caseNumber String @unique title String status String // new, active, resolved, archived priority String clientId String organizationId String assignedTo String? metadata Json createdAt DateTime @default(now()) updatedAt DateTime @updatedAt client Client @relation(fields: [clientId], references: [id]) organization Organization @relation(fields: [organizationId], references: [id]) knowledge CaseKnowledge[] conversations Conversation[] }`

`model Client { id String @id @default(uuid()) name String email String? phone String? organizationId String metadata Json createdAt DateTime @default(now()) updatedAt DateTime @updatedAt organization Organization @relation(fields: [organizationId], references: [id]) cases Case[] }`

`model CaseKnowledge { id String @id @default(uuid()) caseId String type String content Json source String timestamp DateTime @default(now()) case Case @relation(fields: [caseId], references: [id]) }` **Case Isolation:** - Add `caseId` to all case-related tables - Create indexes on `caseId` - Implement row-level security for case data - **Critical:** Ensure case knowledge cannot be accessed without case context

Testing Implementation: - **Case Isolation Tests (CRITICAL):** - Test knowledge from Case A cannot be accessed in Case B - Test knowledge from same client but different cases is isolated - Test cross-case query restrictions - **Case Knowledge Leakage Tests:** - Attempt to retrieve knowledge from different case - Verify access is denied - **Client Management Tests:** - Test client CRUD operations - Test client-case relationships - **Case Context Resolution Tests:** - Test context is correctly resolved for each case - Test context doesn't leak between cases

RING 6 — OBSERVATION (WHATSAPP)

Purpose: Implement WhatsApp observation and conversation understanding

Why Now: Case model is ready, need observation channel

Dependencies: Ring 5

Capabilities Introduced: - WhatsApp Integration [SHARED] - Real-time Capture [GB] - Observation Buffer [GB] - Conversation Understanding [DOMAIN] - Client Identification [SHARED] - Case Resolution [SHARED]

Architecture Impact: - Establishes observation pipeline - Implements async processing - Connects external systems to Brain

Frontend Tasks: - WhatsApp connection UI - Conversation viewer - Observation dashboard - Case assignment UI

Backend Tasks: - WhatsApp integration service - Observation buffer service - Conversation processing service - Client identification service - Case resolution service

AI Tasks: - Conversation understanding - Client identification - Case resolution - Fact extraction - Conversation structure analysis

Data Science Tasks: - Define conversation schema - Create conversation understanding dataset - Define client identification features - Create fact extraction evaluation

Database/Data Tasks: - Create conversation tables - Create observation buffer - Create fact tables - Implement conversation indexing

Integration Tasks: - WhatsApp Business API integration - WhatsApp mock for testing

Testing Tasks: - WhatsApp integration tests - Conversation understanding tests - Client identification tests - Case resolution tests - Observation buffer tests - Async processing tests

Deliverables: - WhatsApp integration working - Real-time capture working - Async processing working - Conversation understanding working - Client identification working - Case resolution working - Observation buffer working

Definition of Done: - Can receive WhatsApp messages in real-time - Messages are buffered and processed asynchronously - Can identify clients from conversations - Can resolve to existing or new cases - Can understand conversation structure - Can extract facts - Case context is maintained - Observation buffer handles load

Risks: - WhatsApp API limitations - Async processing complexity - Client identification accuracy - Case resolution errors

What the Next Ring Unlocks: - Reasoning and assistance can be provided

IMPLEMENTATION GUIDANCE & TOOLS

Frontend Implementation: - **Tools:** React, TypeScript, Socket.io-client, React Chat Elements - **Setup:** `bash npm install @chatscope/chat-ui-kit-react` - **Components to Create:** - WhatsAppConnectionUI.tsx - Connect WhatsApp Business API - ConversationViewer.tsx - Display conversation threads - ObservationDashboard.tsx - Real-time observation monitoring - CaseAssignmentUI.tsx - Assign conversations to cases - **Real-time Features:** - Live message feed via WebSocket - Conversation status indicators - Case assignment interface

Backend Implementation: - **Tools:** Express, WhatsApp Business API SDK, Bull (job queue), Redis - **Setup:** `bash npm install whatsapp-web.js npm install bull redis` - **Services to Create:** -

WhatsAppIntegrationService.ts - WhatsApp API integration - ObservationBufferService.ts - Async observation buffering - ConversationProcessingService.ts - Process conversations - ClientIdentificationService.ts - Identify clients from conversations - CaseResolutionService.ts - Resolve to existing or new cases - **API Endpoints:** - POST /api/whatsapp/connect - Connect WhatsApp - POST /api/whatsapp/disconnect - Disconnect WhatsApp - GET /api/conversations - List conversations - POST /api/conversations/assign-case - Assign to case - **Async Processing:** - Use Bull job queue for message processing - Use Redis for job queue storage - Implement retry logic for failed jobs

AI Implementation: - **Tools:** OpenAI API, LangChain - **Services to Create:** - ConversationUnderstandingAI.ts - Understand conversation structure - ClientIdentificationAI.ts - Identify clients from messages - CaseResolutionAI.ts - Resolve to existing or new case - FactExtractionAI.ts - Extract facts from conversations - ConversationStructureAI.ts - Analyze conversation flow - **Prompt Templates:** - Create conversation analysis prompts - Create client identification prompts - Create fact extraction prompts - **Conversation Analysis:** - Use LLM to understand conversation intent - Extract key information (dates, amounts, people) - Identify legal issues mentioned

Data Science Implementation: - **Tools:** Python, Pandas, Scikit-learn - **Tasks:** - Define conversation schema - Create conversation understanding dataset - Define client identification features - Create fact extraction evaluation - **Jupyter Notebooks:** - 01_conversation_schema.ipynb - Design conversation data structure - 02_conversation_analysis.ipynb - Analyze conversation patterns - 03_client_id_features.ipynb - Design client identification features - 04_fact_extraction_eval.ipynb - Evaluate fact extraction

Database Implementation: - **Prisma Schema Extensions:** `prisma model Conversation { id String @id @default(uuid()) source String // whatsapp, email, phone sourceId String // WhatsApp message ID caseId String? clientId String? status String // processing, analyzed, assigned metadata Json createdAt DateTime @default(now()) updatedAt DateTime @updatedAt case Case? @relation(fields: [caseId], references: [id]) client Client? @relation(fields: [clientId], references: [id]) messages Message[] }`

`model Message { id String @id @default(uuid()) conversationId String sender String // client, lawyer content String timestamp DateTime metadata Json conversation Conversation @relation(fields: [conversationId], references: [id]) }`

`model Fact { id String @id @default(uuid()) conversationId String type String content String confidence Float timestamp DateTime @default(now()) conversation Conversation @relation(fields: [conversationId], references: [id]) }` - **Observation Buffer:** - Create observation buffer table - Implement FIFO queue for processing - Add indexes on status, timestamp - **Conversation Indexing:** - Create full-text search index on message content - Create vector index for semantic search

Integration Implementation: - **WhatsApp Business API:** - Register for WhatsApp Business API - Configure webhook endpoint - Implement webhook handler - Handle message events - **WhatsApp Mock for Testing:** - Create mock WhatsApp service for local testing - Simulate incoming messages - Test conversation processing

Testing Implementation: - **WhatsApp Integration Tests:** - Test webhook receives messages - Test message parsing - Test real-time capture - **Conversation Understanding Tests:** - Test conversation structure analysis - Test intent recognition - **Client Identification Tests:** - Test client identification accuracy - Test new client vs existing client - **Case Resolution Tests:** - Test resolution to existing case - Test creation of new case - **Observation Buffer Tests:** - Test buffer handles high volume - Test async processing - Test job queue performance

RING 7 — REASONING & ASSISTANCE

Purpose: Implement reasoning, assistance, and lawyer support

Why Now: Observations are flowing, need to provide value

Dependencies: Ring 6

Capabilities Introduced: - Reasoning Engine [GB] - Legal Reasoning [DOMAIN] - Assistance Generation [GB] - Similar Case Retrieval [DOMAIN] - Conflict Detection [GB] - Missing Information Identification [DOMAIN]

Architecture Impact: - Implements core intelligence value - Establishes assistance patterns - Enables lawyer productivity

Frontend Tasks: - Assistance interface - Similar cases viewer - Conflict alerts UI - Missing information UI - Lawyer dashboard

Backend Tasks: - Reasoning service - Assistance service - Similar case service - Conflict detection service - Missing information service

AI Tasks: - Reasoning implementation - Legal reasoning - Assistance generation - Similar case algorithms - Conflict detection - Missing information identification

Data Science Tasks: - Create reasoning evaluation dataset - Define assistance quality metrics - Create similar case evaluation - Define conflict detection rules - Create missing information evaluation

Database/Data Tasks: - Create assistance tables - Create conflict tables - Create similar case indexes

Integration Tasks: - None

Testing Tasks: - Reasoning evaluation - Assistance quality tests - Similar case retrieval tests - Conflict detection tests - Missing information tests - End-to-end assistance tests

Deliverables: - Reasoning working - Legal reasoning working - Assistance generation working - Similar case retrieval working - Conflict detection working - Missing information identification working - Assistance quality meets baseline

Definition of Done: - Can reason over case context - Can provide legal assistance - Can find similar cases - Can detect conflicts - Can identify missing information - Assistance is useful to lawyers - Assistance quality meets evaluation metrics

Risks: - Poor reasoning quality - Hallucinations - False conflicts - Poor assistance quality

What the Next Ring Unlocks: - Learning and validation can be implemented

IMPLEMENTATION GUIDANCE & TOOLS

Frontend Implementation: - **Tools:** React, TypeScript, React Markdown, React Syntax Highlighter - **Setup:** `bash npm install react-markdown react-syntax-highlighter` - **Components to Create:** - `AssistanceInterface.tsx` - Display AI assistance with citations - `SimilarCasesViewer.tsx` - Show similar cases with relevance scores - `ConflictAlertsUI.tsx` - Display detected conflicts - `MissingInformationUI.tsx` - Show missing information suggestions - `LawyerDashboard.tsx` - Main lawyer workspace - **Assistance UI Features:** - Markdown rendering for AI responses - Citation links to source documents - Accept/Reject feedback on assistance - Conflict highlighting

Backend Implementation: - **Tools:** Express, Prisma, LangChain - **Services to Create:** - `ReasoningService.ts` - Multi-step reasoning orchestration - `AssistanceService.ts` - Generate assistance responses - `SimilarCaseService.ts` - Find similar cases - `ConflictDetectionService.ts` - Detect conflicts in knowledge - `MissingInformationService.ts` - Identify missing information - **API Endpoints:** - `POST /api/reasoning/query` - Submit reasoning query - `POST /api/assistance/generate` - Generate assistance - `GET /api/cases/:id/similar` - Find similar cases - `GET /api/conflicts/check` - Check for conflicts - `GET /api/missing-info/:caseId` - Get missing information - **Reasoning Orchestration:** - Use LangChain chains for multi-step reasoning - Implement citation generation - Track reasoning steps for transparency

AI Implementation: - **Tools:** OpenAI API, LangChain, LangSmith (tracing) - **Services to Create:** - `ReasoningAI.ts` - Multi-step reasoning with citations - `LegalReasoningAI.ts` - Legal-specific reasoning

patterns - AssistanceGenerationAI.ts - Generate helpful assistance - SimilarCaseAI.ts - Find semantically similar cases - ConflictDetectionAI.ts - Detect knowledge conflicts - MissingInformationAI.ts - Identify missing information - **Prompt Templates:** - Create reasoning prompts with citation requirements - Create legal reasoning prompts - Create assistance generation prompts - Create conflict detection prompts - **Reasoning Strategy:** - Use chain-of-thought prompting - Implement citation generation - Configure confidence scoring - Use LangSmith for tracing and debugging

Data Science Implementation: - **Tools:** Python, Pandas, Scikit-learn - **Tasks:** - Create reasoning evaluation dataset - Define assistance quality metrics - Create similar case evaluation - Define conflict detection rules - Create missing information evaluation - **Jupyter Notebooks:** - 01_reasoning_eval.ipynb - Evaluate reasoning quality - 02_assistance_metrics.ipynb - Define assistance quality metrics - 03_similar_case_eval.ipynb - Evaluate case similarity - 04_conflict_detection.ipynb - Test conflict detection - 05_missing_info_eval.ipynb - Evaluate missing information detection

Database Implementation: - **Prisma Schema Extensions:** `prisma model Assistance { id String @id @default(uuid()) caseId String query String response Text citations Json confidence Float feedback String? // helpful, not-helpful timestamp DateTime @default(now()) case Case @relation(fields: [caseId], references: [id]) }`

`model SimilarCase { id String @id @default(uuid()) caseId String similarCaseId String similarity Float reason String timestamp DateTime @default(now()) }`

`model Conflict { id String @id @default(uuid()) knowledgeId String conflictType String description String resolved Boolean timestamp DateTime @default(now()) }`

`model MissingInfo { id String @id @default(uuid()) caseId String type String description String priority String timestamp DateTime @default(now()) case Case @relation(fields: [caseId], references: [id]) }` -

Indexes: - Create indexes on caseId for assistance - Create vector index for similar case search - Create indexes on conflict status

Testing Implementation: - **Reasoning Evaluation:** - Create evaluation dataset with known answers - Measure reasoning accuracy - Evaluate citation quality - **Assistance Quality Tests:** - Test assistance is helpful to lawyers - Measure lawyer satisfaction - Test assistance improves over time - **Similar Case Retrieval Tests:** - Test similar case accuracy - Measure relevance scores - **Conflict Detection Tests:** - Test conflict detection accuracy - Test false positive rate - **Missing Information Tests:** - Test missing information identification - Test priority scoring - **End-to-End Assistance Tests:** - Test complete assistance flow - Test lawyer feedback loop

RING 8 — LEARNING & VALIDATION

Purpose: Implement learning loop and human validation

Why Now: Assistance is working, need to capture learning

Dependencies: Ring 7

Capabilities Introduced: - Learning Engine [GB] - Validation / Governance Engine [GB] - Knowledge Candidate Generation [GB] - Human Validation Workflow [ORG] - Learning Signals [GB] - Knowledge Promotion [GB]

Architecture Impact: - Implements core learning loop - Establishes human governance - Enables continuous improvement

Frontend Tasks: - Validation queue UI - Knowledge candidate viewer - Validation interface - Learning dashboard - Knowledge history viewer

Backend Tasks: - Learning service - Validation service - Knowledge candidate service - Learning signal service - Knowledge promotion service

AI Tasks: - Learning signal generation - Knowledge candidate generation - Learning from validation - Knowledge scoring

Data Science Tasks: - Define learning metrics - Create learning evaluation - Define knowledge scoring - Create validation analysis

Database/Data Tasks: - Create validation queue tables - Create knowledge candidate tables - Create learning signal tables - Implement knowledge promotion logic

Integration Tasks: - None

Testing Tasks: - Learning loop tests - Validation workflow tests - Knowledge promotion tests - Learning signal tests - End-to-end learning tests

Deliverables: - Learning loop working - Knowledge candidate generation working - Human validation workflow working - Learning signals working - Knowledge promotion working - Learning is measurable

Definition of Done: - Can generate knowledge candidates - Validation workflow is functional - Humans can validate candidates - Validated knowledge is promoted - Learning signals are captured - Learning improves future assistance - Learning is measurable

Risks: - Poor candidate quality - Validation bottleneck - Learning instability - Knowledge contamination

What the Next Ring Unlocks: - Insights and governance can be implemented

IMPLEMENTATION GUIDANCE & TOOLS

Frontend Implementation: - **Tools:** React, TypeScript, React Query, React Virtualized (for large lists) - **Setup:** `bash npm install @tanstack/react-query react-virtualized` - **Components to Create:** - `ValidationQueueUI.tsx` - Display knowledge candidates for validation - `KnowledgeCandidateViewer.tsx` - View candidate details - `ValidationInterface.tsx` - Approve/reject/modify candidates - `LearningDashboard.tsx` - Show learning metrics and trends - `KnowledgeHistoryViewer.tsx` - View knowledge evolution - **Validation UI Features:** - Batch validation interface - Confidence score visualization - Source document preview - Validation history timeline

Backend Implementation: - **Tools:** Express, Prisma, Bull (job queue) - **Services to Create:** - `LearningService.ts` - Learning signal generation and processing - `ValidationService.ts` - Validation workflow management - `KnowledgeCandidateService.ts` - Candidate generation and management - `LearningSignalService.ts` - Capture learning signals - `KnowledgePromotionService.ts` - Promote validated knowledge - **API Endpoints:** - `GET /api/validation/queue` - Get validation queue - `POST /api/validation/approve/:id` - Approve candidate - `POST /api/validation/reject/:id` - Reject candidate - `POST /api/validation/modify/:id` - Modify candidate - `GET /api/learning/metrics` - Get learning metrics - `GET /api/knowledge/history/:id` - Get knowledge history - **Learning Loop:** - Implement candidate generation triggers - Implement validation workflow - Implement knowledge promotion logic - Track learning signals

AI Implementation: - **Tools:** OpenAI API, LangChain - **Services to Create:** - `LearningSignalAI.ts` - Generate learning signals - `KnowledgeCandidateAI.ts` - Generate knowledge candidates - `LearningFromValidationAI.ts` - Learn from validation decisions - `KnowledgeScoringAI.ts` - Score knowledge candidates - **Prompt Templates:** - Create knowledge candidate generation prompts - Create learning signal extraction prompts - Create knowledge scoring prompts - **Learning Strategy:** - Use validation feedback to improve future assistance - Track which knowledge is most useful - Identify knowledge gaps

Data Science Implementation: - **Tools:** Python, Pandas, Scikit-learn, Matplotlib - **Tasks:** - Define learning metrics - Create learning evaluation - Define knowledge scoring - Create validation analysis - **Jupyter Notebooks:** - `01_learning_metrics.ipynb` - Define learning metrics - `02_learning_eval.ipynb` - Evaluate learning effectiveness - `03_knowledge_scoring.ipynb` - Design knowledge scoring -

04_validation_analysis.ipynb - Analyze validation patterns - 05_learning_trends.ipynb - Visualize learning trends

Database Implementation: - Prisma Schema Extensions: ```prisma model KnowledgeCandidate { id String @id @default(uuid()) type String content Json source String confidence Float status String // pending, approved, rejected validatorId String? validated DateTime? feedback String? timestamp DateTime @default(now()) }

model LearningSignal { id String @id @default(uuid()) type String source String strength Float metadata Json timestamp DateTime @default(now()) }

model ValidationHistory { id String @id @default(uuid()) knowledgeId String validatorId String action String // approve, reject, modify reason String? timestamp DateTime @default(now()) } ``` - **Validation Queue:** - Create validation queue table - Implement priority queue (high confidence first) - Add indexes on status, confidence - **Learning Tracking:** - Track learning signals over time - Track validation patterns - Track knowledge promotion history

Testing Implementation: - Learning Loop Tests: - Test complete learning loop - Test candidate generation - Test validation workflow - Test knowledge promotion - **Validation Workflow Tests:** - Test validation queue management - Test batch validation - Test validation history tracking - **Knowledge Promotion Tests:** - Test promotion logic - Test knowledge state transitions - **Learning Signal Tests:** - Test learning signal generation - Test learning signal capture - **End-to-End Learning Tests:** - Test learning improves future assistance - Test learning is measurable

RING 9 — INSIGHTS & GOVERNANCE

Purpose: Implement insights, audit, and governance dashboard

Why Now: Learning is working, need visibility and control

Dependencies: Ring 8

Capabilities Introduced: - Insight Engine [GB] - Audit Engine [GB] - Governance Dashboard [ORG] - Organization Intelligence [ORG] - Activity Monitoring [GB] - Autonomy Controller [GB]

Architecture Impact: - Provides organizational visibility - Establishes governance oversight - Enables management insights

Frontend Tasks: - Governance dashboard - Insight dashboard - Activity viewer - Audit log viewer - Autonomy configuration UI

Backend Tasks: - Insight service - Audit service - Governance service - Activity monitoring service - Autonomy service

AI Tasks: - Insight generation - Activity pattern recognition - Anomaly detection

Data Science Tasks: - Define insight metrics - Create insight evaluation - Define activity patterns - Create anomaly detection models

Database/Data Tasks: - Create audit log tables - Create activity tables - Create insight tables - Implement aggregation queries

Integration Tasks: - None

Testing Tasks: - Insight generation tests - Audit logging tests - Governance tests - Activity monitoring tests - Autonomy control tests

Deliverables: - Insights working - Audit logging working - Governance dashboard working - Activity monitoring working - Autonomy control working - Insights are useful

Definition of Done: - Can generate organizational insights - Audit trail is complete - Governance dashboard is functional - Activity is monitored - Autonomy can be controlled - Insights are actionable

Risks: - Poor insight quality - Audit complexity - Governance overhead

What the Next Ring Unlocks: - Complete Legal V1 integration

IMPLEMENTATION GUIDANCE & TOOLS

Frontend Implementation: - **Tools:** React, TypeScript, Recharts, React Query - **Setup:** `bash npm install recharts @tanstack/react-query date-fns` - **Components to Create:** - `GovernanceDashboard.tsx` - Main governance dashboard - `InsightDashboard.tsx` - Display organizational insights - `ActivityViewer.tsx` - View system activity logs - `AuditLogViewer.tsx` - View audit trail - `AutonomyConfigurationUI.tsx` - Configure autonomy levels - **Dashboard Features:** - Time-series charts for activity - Insight cards with trends - Activity timeline - Autonomy level controls

Backend Implementation: - **Tools:** Express, Prisma, Bull (scheduled jobs) - **Services to Create:** - `InsightService.ts` - Generate organizational insights - `AuditService.ts` - Log and retrieve audit trail - `GovernanceService.ts` - Governance operations - `ActivityMonitoringService.ts` - Monitor system activity - `AutonomyService.ts` - Manage autonomy levels - **API Endpoints:** - `GET /api/insights` - Get organizational insights - `GET /api/audit/logs` - Get audit logs - `GET /api/governance/status` - Get governance status - `GET /api/activity` - Get system activity - `PUT /api/autonomy/configure` - Configure autonomy - **Scheduled Jobs:** - Use Bull to schedule insight generation - Run daily/weekly insight reports - Aggregate activity metrics

AI Implementation: - **Tools:** OpenAI API, LangChain - **Services to Create:** - `InsightGenerationAI.ts` - Generate insights from activity - `ActivityPatternAI.ts` - Recognize activity patterns - `AnomalyDetectionAI.ts` - Detect anomalies in activity - **Prompt Templates:** - Create insight generation prompts - Create pattern recognition prompts - Create anomaly detection prompts - **Insight Generation:** - Analyze activity patterns - Identify trends - Detect anomalies - Generate actionable insights

Data Science Implementation: - **Tools:** Python, Pandas, Scikit-learn, Prophet (time series) - **Setup:** `bash pip install prophet statsmodels` - **Tasks:** - Define insight metrics - Create insight evaluation - Define activity patterns - Create anomaly detection models - **Jupyter Notebooks:** - `01_insight_metrics.ipynb` - Define insight metrics - `02_insight_eval.ipynb` - Evaluate insight quality - `03_activity_patterns.ipynb` - Analyze activity patterns - `04_anomaly_detection.ipynb` - Build anomaly detection models

Database Implementation: - **Prisma Schema Extensions:** ```prisma model Insight { id String @id @default(uuid()) type String title String description String metrics Json priority String timestamp DateTime @default(now()) }`

`model AuditLog { id String @id @default(uuid()) userId String action String resource String details Json timestamp DateTime @default(now()) }`

`model Activity { id String @id @default(uuid()) type String description String metadata Json timestamp DateTime @default(now()) }`

`model AutonomyConfig { id String @id @default(uuid()) organizationId String roleId String level String // observe, assist, execute, automate permissions Json timestamp DateTime @default(now()) }` - **Audit Trail:** - Create audit log table - Implement audit logging middleware - Add indexes on `userId`, `action`, `timestamp` - **Activity Monitoring:** - Create activity table - Implement activity tracking - Add indexes on `type`, `timestamp`

Testing Implementation: - **Insight Generation Tests:** - Test insight generation accuracy - Test insight relevance - **Audit Logging Tests:** - Test audit trail completeness - Test audit log retrieval - **Governance Tests:** - Test governance dashboard functionality - Test governance controls - **Activity Monitoring Tests:** -

Test activity tracking - Test activity pattern recognition - **Autonomy Control Tests:** - Test autonomy configuration - Test autonomy enforcement

RING 10 — LEGAL V1

Purpose: Integrate all components into complete Legal V1

Why Now: All components are ready, need end-to-end integration

Dependencies: Ring 9

Capabilities Introduced: - Complete Legal Intelligence [SHARED] - End-to-End Flow [SHARED] - V1 Demo [SHARED] - Evaluation [SHARED]

Architecture Impact: - Validates complete architecture - Demonstrates end-to-end value - Proves platform vision

Frontend Tasks: - Complete Legal UI - End-to-end user flows - Demo preparation

Backend Tasks: - Complete Legal APIs - End-to-end orchestration - Performance optimization

AI Tasks: - Complete Legal AI - End-to-end reasoning - Performance optimization

Data Science Tasks: - Complete Legal evaluation - Unseen evaluation set - Final metrics - Documentation

Database/Data Tasks: - Data optimization - Index optimization - Backup strategy

Integration Tasks: - WhatsApp integration - CRM integration - Document system integration

Testing Tasks: - Complete end-to-end tests - Unseen evaluation - Performance tests - Security tests - User acceptance tests

Deliverables: - Complete Legal V1 working end-to-end - V1 demo - Evaluation report - Documentation - Architecture validated

Definition of Done: - Complete end-to-end flow works - Client contacts organization -> Brain observes -> Case created -> Assistance provided -> Learning happens - Unseen evaluation meets metrics - Demo is impressive - Architecture is validated - Documentation is complete - Team is ready for next domain

Risks: - Integration complexity - Performance issues - Evaluation failure - Demo failure

What the Next Ring Unlocks: - Future domain expansion (Sales, HR, etc.)

IMPLEMENTATION GUIDANCE & TOOLS

Frontend Implementation: - **Tools:** React, TypeScript, React Query, Framer Motion (animations) - **Setup:** `bash npm install framer-motion @tanstack/react-query` - **Components to Create:** - `CompleteLegalUI.tsx` - Integrated Legal interface - `EndToEndFlowUI.tsx` - Demonstrate complete user flow - `DemoPreparationUI.tsx` - Demo mode interface - **Demo Features:** - Guided tour of complete flow - Demo data for showcase - Performance metrics display - Animated transitions

Backend Implementation: - **Tools:** Express, Prisma, Redis (caching), Nginx (reverse proxy) - **Services to Create:** - `CompleteLegalService.ts` - Orchestrate all Legal V1 services - `EndToEndOrchestrationService.ts` - Coordinate complete flow - `PerformanceOptimizationService.ts` - Optimize performance - **API Endpoints:** - `POST /api/demo/start` - Start demo mode - `GET /api/demo/status` - Get demo status - `POST /api/demo/reset` - Reset demo - **Performance Optimization:** - Implement response caching with Redis - Optimize database queries - Implement connection pooling - Add rate limiting

AI Implementation: - **Tools:** OpenAI API, LangChain, LangSmith (monitoring) - **Services to Create:** - CompleteLegalAI.ts - Orchestrate all Legal AI services - EndToEndReasoningAI.ts - End-to-end reasoning - **Optimization:** - Cache LLM responses - Batch API calls - Use streaming for long responses - Monitor token usage with LangSmith

Data Science Implementation: - **Tools:** Python, Pandas, Scikit-learn - **Tasks:** - Complete Legal evaluation - Create unseen evaluation set - Final metrics calculation - Documentation - **Jupyter Notebooks:** - 01_complete_eval.ipynb - Complete Legal evaluation - 02_unseen_eval.ipynb - Unseen evaluation set - 03_final_metrics.ipynb - Final metrics calculation - 04_documentation.ipynb - Generate documentation

Database Implementation: - **Optimization:** - Create composite indexes for common queries - Optimize vector indexes - Implement database connection pooling - Add database backups - **Backup Strategy:** - Automated daily backups - Point-in-time recovery - Backup verification

Integration Implementation: - **WhatsApp Integration:** - Complete WhatsApp Business API integration - Test webhook reliability - Implement error handling - **CRM Integration:** - Implement CRM adapter - Test data synchronization - Handle CRM errors gracefully - **Document System Integration:** - Integrate with document storage - Implement document versioning - Handle large files

Testing Implementation: - **End-to-End Tests:** - Test complete user flow: Client contacts -> Brain observes -> Case created -> Assistance provided -> Learning happens - Test all components integrated - Test error handling - **Unseen Evaluation:** - Create unseen evaluation set - Test on unseen legal scenarios - Measure performance on unseen data - **Performance Tests:** - Load testing with k6 or Artillery - Response time benchmarks - Throughput benchmarks - **Security Tests:** - Penetration testing - Permission escalation tests - Data leakage tests - **User Acceptance Tests:** - Test with real lawyers - Collect feedback - Iterate based on feedback

Demo Preparation: - **Demo Script:** - Create step-by-step demo script - Prepare demo data - Practice demo flow - **Demo Environment:** - Set up demo-specific environment - Use demo data (not real client data) - Ensure demo is reliable

RING 11+ — FUTURE DOMAINS

Purpose: Add new domains without rebuilding the Brain

Why Now: Legal V1 proves architecture works

Dependencies: Ring 10

Capabilities Introduced: - Sales Domain [DOMAIN] - HR Domain [DOMAIN] - Other domains [DOMAIN]

Architecture Impact: - Validates platform reusability - Expands market - Scales platform

Frontend Tasks: - Domain-specific UIs - Domain configuration

Backend Tasks: - Domain services - Domain adapters

AI Tasks: - Domain-specific reasoning - Domain-specific engines

Data Science Tasks: - Domain data acquisition - Domain evaluation - Domain metrics

Database/Data Tasks: - Domain knowledge loading - Domain indexes

Integration Tasks: - Domain-specific integrations

Testing Tasks: - Domain tests - Cross-domain isolation tests

Deliverables: - New domain working - Platform reusability validated - Multi-domain deployment possible

Definition of Done: - New domain works end-to-end - General Brain is reused - Domain isolation is maintained - Platform reusability is proven

Risks: - Domain complexity - Cross-domain contamination - Platform limitations

IMPLEMENTATION GUIDANCE & TOOLS

Frontend Implementation: - **Tools:** React, TypeScript, React Query - **Components to Create:** - DomainSpecificUI.tsx - Generic domain UI template - DomainConfigurationUI.tsx - Configure new domain - **Domain UI Template:** - Create reusable domain UI components - Configure domain-specific fields - Customize domain workflows

Backend Implementation: - **Tools:** Express, Prisma - **Services to Create:** - DomainService.ts - Generic domain service - DomainAdapterService.ts - Domain adapter pattern - **API Endpoints:** - POST /api/domains/create - Create new domain - GET /api/domains/:id/config - Get domain configuration - PUT /api/domains/:id/config - Update domain configuration - **Domain Adapter Pattern:** - Create domain adapter interface - Implement domain-specific adapters - Reuse General Brain services

AI Implementation: - **Tools:** OpenAI API, LangChain - **Services to Create:** - DomainSpecificReasoningAI.ts - Domain-specific reasoning - DomainEngineAI.ts - Domain-specific engines - **Prompt Templates:** - Create domain-specific prompt templates - Configure domain terminology - **Domain Adaptation:** - Use domain-specific prompts - Configure domain knowledge base - Reuse General Brain reasoning

Data Science Implementation: - **Tools:** Python, Pandas, Scikit-learn - **Tasks:** - Domain data acquisition - Domain evaluation - Domain metrics - **Jupyter Notebooks:** - 01_domain_data.ipynb - Acquire domain data - 02_domain_eval.ipynb - Evaluate domain performance - 03_domain_metrics.ipynb - Define domain-specific metrics

Database Implementation: - **Prisma Schema Extensions:** ```prisma model Domain { id String @id @default(uuid()) name String type String // sales, hr, finance, etc. config Json // domain-specific configuration knowledge Json // domain knowledge createdAt DateTime @default(now()) updatedAt DateTime @updatedAt }

model DomainKnowledge { id String @id @default(uuid()) domainId String type String content Json embedding Unsupported("vector(1536)")? timestamp DateTime @default(now()) domain Domain @relation(fields: [domainId], references: [id]) } ``` - **Domain Isolation:** - Store domain knowledge with domainId - Ensure domain isolation - Create domain-specific indexes

Integration Implementation: - **Domain-Specific Integrations:** - Implement domain-specific adapters - Configure domain-specific APIs - Handle domain-specific data formats

Testing Implementation: - **Domain Tests:** - Test domain-specific functionality - Test domain isolation - **Cross-Domain Isolation Tests:** - Verify data doesn't leak between domains - Test cross-domain contamination

Domain Addition Process: 1. **Acquire Domain Knowledge:** - Collect domain documents - Clean and normalize data - Create domain taxonomy

2. Define Domain Schema:

- Design domain-specific data structures
- Define domain workflows
- Configure domain terminology

3. Implement Domain Adapter:

- Create domain adapter service
- Implement domain-specific AI
- Configure domain prompts

4. Create Domain Evaluation:

- Create domain-specific evaluation dataset
- Define domain metrics
- Test domain performance

5. Load Domain Knowledge:

- Load domain corpus into knowledge stores
- Create domain-specific indexes
- Configure domain retrieval

6. Configure Domain:

- Configure domain UI
- Configure domain workflows
- Configure domain permissions

7. Test Domain Isolation:

- Verify domain isolation
- Test cross-domain contamination

8. Deploy Domain:

- Deploy to organization
 - Monitor domain performance
 - Iterate based on feedback
-

TEAM RESPONSIBILITIES

AI TEAM

Responsibilities: - AI / LLM engineering - Reasoning implementation - Knowledge extraction - Retrieval algorithms - Agent/Brain logic - Learning logic - AI evaluation

Ring Responsibilities: | Ring | AI Tasks | |——|—————| | 0 | Basic LLM setup, prompt management | | 1 | Perception, context resolution, basic retrieval | | 2 | Knowledge extraction, embeddings, retrieval algorithms, RAG | | 3 | Legal retrieval, legal classification, legal document understanding | | 4 | Organization context resolution, permission-aware reasoning | | 5 | Case context resolution, case-specific reasoning | | 6 | Conversation understanding, client identification, fact extraction | | 7 | Reasoning, legal reasoning, assistance, similar cases, conflicts | | 8 | Learning signals, knowledge candidate generation, learning from validation | | 9 | Insight generation, activity patterns, anomaly detection | | 10 | Complete Legal AI, end-to-end reasoning | | 11+ | Domain-specific reasoning, domain engines |

DATA SCIENCE TEAM

Responsibilities: - Data acquisition - Data cleaning - Legal corpus preparation - Taxonomy - ML experiments - Evaluation datasets - Statistical evaluation - Domain-specific ML where justified

Ring Responsibilities: | Ring | Data Science Tasks | |——|—————| | 0 | Data pipeline infrastructure, evaluation framework | | 1 | Observation schema, context schema, knowledge object schema | | 2 | Knowledge taxonomy, embedding strategy, retrieval evaluation | | 3 | Legal corpus acquisition, cleaning, taxonomy, classification dataset | | 4 | Organization schema, role taxonomy, permission model | | 5 | Case schema, client schema, case knowledge schema | | 6 | Conversation schema, understanding dataset, client ID features | | 7 | Reasoning evaluation, assistance metrics, similar case eval | | 8 | Learning metrics, knowledge scoring, validation analysis | | 9 | Insight metrics, activity patterns, anomaly detection models | | 10 | Complete Legal evaluation, unseen evaluation, final metrics | | 11+ | Domain data acquisition, domain evaluation, domain metrics |

BACKEND TEAM

Responsibilities: - APIs - Services - Database - Authentication - Authorization - Event processing - Integrations - Persistence - Infrastructure

Ring Responsibilities: | Ring | Backend Tasks | |——|———| | 0 | API skeleton, database connections, auth skeleton | | 1 | Perception, observation, context, memory, retrieval services | | 2 | Knowledge, multi-store coordination, provenance, advanced retrieval | | 3 | Legal domain, legal retrieval, legal classification, legal doc understanding | | 4 | Organization, roles, permissions, validation authority, org context | | 5 | Case, client, case isolation, case knowledge services | | 6 | WhatsApp integration, observation buffer, conversation processing | | 7 | Reasoning, assistance, similar case, conflict, missing info services | | 8 | Learning, validation, knowledge candidate, learning signal, promotion | | 9 | Insight, audit, governance, activity monitoring, autonomy services | | 10 | Complete Legal APIs, orchestration, optimization | | 11+ | Domain services, domain adapters |

FRONTEND TEAM

Responsibilities: - Product UI - Dashboards - Case workspace - Brain interfaces - Review/validation interfaces - Interaction flows - Responsive experience

Ring Responsibilities: | Ring | Frontend Tasks | |——|———| | 0 | UI skeleton, routing, Tailwind setup | | 1 | Brain dashboard, observation viewer, context switcher | | 2 | Knowledge viewer, knowledge editor, provenance viewer | | 3 | Legal domain UI, legal document viewer, legal search | | 4 | Org setup wizard, role/permission management, validation authority config | | 5 | Case workspace, client management, case viewer, case knowledge viewer | | 6 | WhatsApp connection UI, conversation viewer, observation dashboard | | 7 | Assistance interface, similar cases viewer, conflict alerts, missing info | | 8 | Validation queue, knowledge candidate viewer, validation interface, learning dashboard | | 9 | Governance dashboard, insight dashboard, activity viewer, audit log, autonomy config | | 10 | Complete Legal UI, end-to-end flows, demo preparation | | 11+ | Domain UIs, domain configuration |

SHARED RESPONSIBILITIES

Cross-Team Coordination: - API contracts - Data schemas - Evaluation metrics - Integration points - Testing strategy - Documentation

CRITICAL PATH

Critical Capabilities

These capabilities block everything else:

1. **Architecture** - Must be defined before implementation
2. **Identity** - Organization, domain, case identification
3. **Organization Isolation** - Security and correctness
4. **Brain Context** - Context resolution engine
5. **Knowledge Storage** - Multi-store architecture
6. **Retrieval** - Basic retrieval capability
7. **Provenance** - Knowledge source tracking
8. **Permissions** - Access control
9. **Event/Observation Infrastructure** - Async processing
10. **Case Model** - First-class object with isolation

Critical Path Sequence

Architecture Definition (Ring 0)

▼

Identity & Isolation (Ring 4)

▼

Brain Context (Ring 1)

▼

Knowledge Storage (Ring 2)

▼

Retrieval (Ring 2)

V

Provenance (Ring 2)

V

Permissions (Ring 4)

V

Observation Infrastructure (Ring 6)

V

Case Model (Ring 5)

RISKS

Major Risks

Risk	Impact	Mitigation	Ring Addressed
Building too much General Intelligence at once	High	Incremental Rings, focus on V1 needs	Ring 1-2
Overengineering	High	Minimum viable capability per Ring	All Rings
Confusing domain knowledge with organization knowledge	High	Clear classification, strict isolation	Ring 3-4
Treating every observation as knowledge	High	Explicit learning loop, validation gate	Ring 8
Lack of provenance	High	Provenance engine, mandatory tracking	Ring 2
Case contamination	Critical	Strict isolation, isolation tests	Ring 5
Weak evaluation	High	Unseen evaluation set, metrics	Ring 2, 3, 7, 10
Fake learning	High	Validation gate, human governance	Ring 8
Excessive token usage	Medium	Retrieval optimization, caching	Ring 2, 7
Building UI before architecture	High	Architecture first, then implementation	Ring 0
Building disconnected team components	High	Ring-based vertical integration	All Rings
Treating CRM as the Brain	High	Clear separation, integration layer	Ring 6, 10
Creating domain-specific logic in General Brain	High	Classification review, architectural review	All Rings
Postponing architecture decisions	High	Architecture documentation before implementation	Ring 0

ANTI-PATTERNS

Things the Team Must NOT Do

DO NOT: - Build a separate Legal Brain - Rebuild the Brain for each organization - Treat every message as knowledge - Allow uncontrolled learning - Mix cases - Hardcode one organization’s workflow into the General Brain - Build fake AI behavior - Build disconnected demos - Build frontend without backend capability - Build ML models without evaluation - Call an LLM and label it “learning” - Use RAG alone as the definition of the Brain - Treat CRM as organizational memory - Postpone architecture decisions until after implementation - Create domain-specific logic inside General Brain unnecessarily - Skip case isolation - Skip provenance tracking - Skip human validation - Assume full autonomy from day one

V1 SUCCESS CRITERIA

Level 1: General Brain Foundation

Success: - All General Brain engines are implemented - Engines are reusable across domains - Multi-store architecture works - Retrieval works with measurable quality - Context resolution works - Permissions are enforced

Level 2: Legal Domain Proof

Success: - Legal domain knowledge is loaded - Legal-specific engines work - Legal retrieval meets domain-specific metrics - Legal document understanding works - Domain is isolated from General Brain

Level 3: Organization-Specific Intelligence

Success: - Organization can be configured - Roles and permissions work - Case model works with strict isolation - WhatsApp observation works - Assistance is useful to lawyers - Learning loop works - Human validation works - Knowledge improves over time - CRM/system integration works

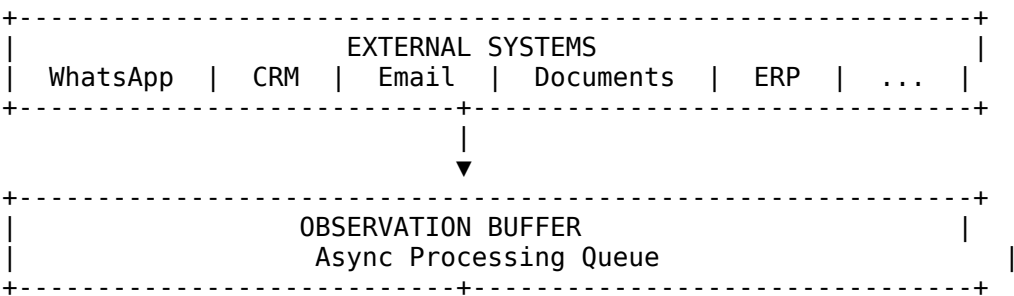
V1 End-to-End Success

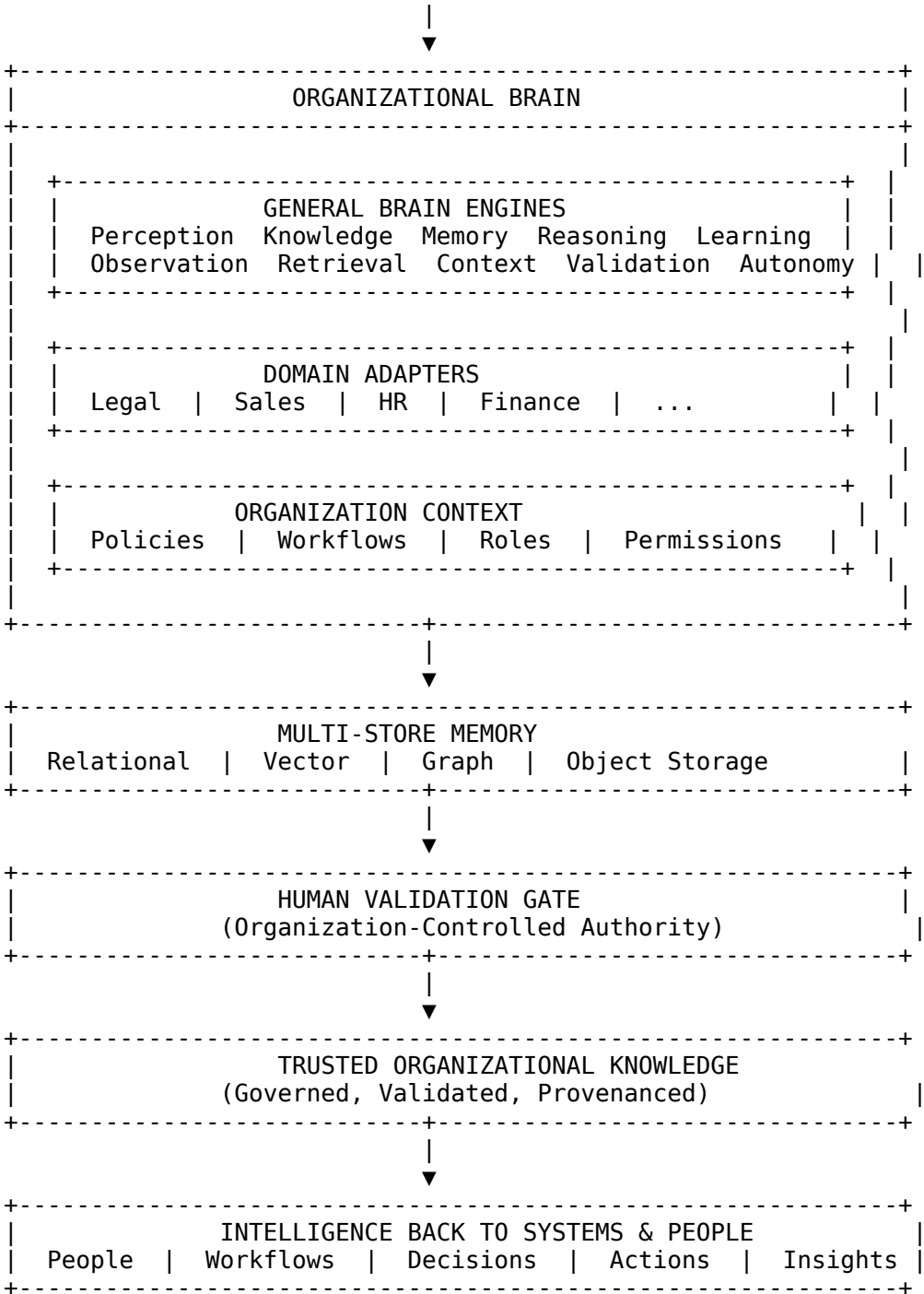
The system demonstrates: - A real General Brain - Real Legal domain knowledge - Real organization-specific context - Real case memory - Real observation - Real retrieval/reasoning - Real human validation - Real learning loop - Real CRM/system integration - Real useful assistance

The scale can be limited. The behavior must be real.

FINAL ARCHITECTURE

Complete System Architecture





Long-Term Product Evolution

- Prototype
- ↓
- Architectural Foundation
- ↓
- General Brain Core
- ↓
- Legal Domain
- ↓
- Organization Deployment
- ↓
- Case Intelligence
- ↓
- Observation
- ↓
- Assistance
- ↓
- Learning
- ↓

Governed Autonomy
V
Validated Legal V1
V
Reusable Platform
V
New Domains (Sales, HR, Finance, ...)
V
Enterprise Intelligence Platform

GLOSSARY

Terms

General Brain [GB] - The reusable intelligence infrastructure built once and used across all domains and organizations.

Domain Intelligence [DOMAIN] - Domain-specific knowledge and capabilities that specialize the General Brain for a particular industry or function.

Organization Intelligence [ORG] - Organization-specific context, knowledge, and configuration that customizes the Brain for a particular organization.

Ring - A complete, working vertical capability that spans Frontend, Backend, AI, Data Science, Database, Integration, and Testing.

Knowledge Candidate - Proposed knowledge awaiting human validation. Not yet trusted.

Trusted Knowledge - Human-validated organizational knowledge. Can be used for reasoning and assistance.

Observation - Raw data captured from systems, people, or documents. Not automatically knowledge.

Case - A first-class object representing a specific matter or engagement. Cases must remain isolated.

Provenance - The chain of sources and validations that tracks where knowledge came from and how it was approved.

Autonomy - The Brain's ability to take action without human intervention. Progresses from Observe to Assist to Execute to Automate.

Governance - The rules and processes that control how the Brain operates, including permissions, validation authority, and oversight.

Validation Authority - The person or role authorized to validate knowledge candidates as trusted organizational knowledge.

AI Employee - A role-specific AI worker built on top of the General Brain, Domain, and Organization Context.

CRM - Customer Relationship Management system. An operational system, not the Brain itself.

WhatsApp Integration - The primary communication channel for Legal V1, enabling real-time client communication.

RAG - Retrieval-Augmented Generation. A technique for enhancing AI responses with retrieved knowledge. Not the entire Brain.

LLM - Large Language Model. The foundational AI technology used for reasoning and generation.

Embedding - Vector representation of text or data used for semantic search and similarity.

Vector Database - Database optimized for storing and querying vector embeddings.

Graph Database - Database optimized for storing and querying relationships and graph structures.

Multi-Store Architecture - Using multiple database types (relational, vector, graph, object) together for different data needs.

ADR - Architecture Decision Record. A document that records important architectural decisions.

Constitution - This document. The highest-level reference for the project.

Invariant - An architectural rule that must not be violated without explicit approval.

Classification - Labeling capabilities as [GB], [DOMAIN], [ORG], or [SHARED] to ensure proper layering.

Isolation - Ensuring data and knowledge from one context (organization, case) does not leak into another.

Tenant - An organization using the Brain. Each tenant must be isolated from others.

Scope Creep - Adding features beyond the defined scope, especially for V1. Must be avoided.

Evaluation Dataset - Data used to test the Brain, kept separate from training data to measure real-world performance.

Unseen Evaluation - Testing on data the Brain has not been trained on, to measure generalization.

Golden Test Cases - Core test scenarios that must always pass to ensure basic functionality.

END OF CONSTITUTION

This document is the authoritative reference for the Organizational Brain project. All development work must align with this Constitution. Any deviation must be documented and approved through the Change Management process.

Generated for the Organizational Brain Development Team Faculty of Computers and Data Science - Graduation Project 2026-2027